

TRAVEL PRO



Submitted by

ABHINAV RAJEEV A : 33223825008

ASHIQ S : 33223825023

GOUTHAM HAREESH : 33223825033

KAILAS JITH V A : 33223825039

SUDEEP S S : 33223825058

Guided by

Mr. Goutham Krishna L.U

Assistant Professor



CHRIST NAGAR COLLEGE

MARANALLOOR, THIRUVANANTHAPURAM

A CMI Educational Institution | Affiliated to the University of Kerala

PROJECT REPORT

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF
BACHELOR OF COMPUTER APPLICATIONS DEGREE OF THE UNIVERSITY OF KERALA

2025



CHRIST NAGAR COLLEGE

MARANALLOOR, THIRUVANANTHAPURAM

A CMI Educational Institution | Affiliated to the University of Kerala



CERTIFICATE

CERTIFIED THAT THIS REPORT TITLED “**TRAVEL PRO**” IS A BONAFIDE RECORD OF THE MAJOR PROJECT WORK DONE BY **ABHINAV RAJEEV A (33223825008)**, **ASHIQ S (33223825023)**, **GOUTHAM HAREESH (33223825033)**, **KAILAS JITH V A (33223825039)** AND **SUDEEP S S (33223825058)** UNDER OUR SUPERVISION AND GUIDANCE AT THE PG DEPARTMENT OF COMPUTER SCIENCE, CHRIST NAGAR COLLEGE, TOWARDS THE PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF **BACHELOR OF COMPUTER APPLICATIONS** OF THE UNIVERSITY OF KERALA.

PRINCIPAL

HEAD OF THE DEPARTMENT

INTERNAL GUIDE

EXTERNAL EXAMINER

1.

2.

ACKNOWLEDGEMENTS

We are using this opportunity to express our sincere gratitude to everyone who supported us throughout the course of this BCA project. We first like to thank the almighty for showering his blessings upon us.

We would like to thank our Manager, **Rev.Fr. Cyriac Madathil CMI** for the encouragement provided.

Then we would like to thank our Principal, **Dr. Jolly Jacob** for the support and encouragement provided.

We express our sincere gratitude towards our Head of the PG Department of Computer Science, **Mrs. Sunitha S Nair** for providing an opportunity for undertaking this project.

We would also like to thank our project guide, **Mr. Goutham Krishna L.U**, Assistant Professor, for all the support given by him throughout the completion of the project.

We express our warm thanks to our external project guide **Mr. Anoop Nockson**, project manager for his support and guidance at Ainsoft Solutions Private Ltd, Trivandrum.

Last but not least we would like to express our profound thanks to all our friends for their support. We thank all the above-mentioned people for their aspiring guidance, invaluable constructive criticism and friendly advice during the project work. We are sincerely grateful to them for sharing their truthful and illuminating views on a number of issues related to the project.

WITH GRATITUDE,

Abhinav Rajeev A

Ashiq S

Goutham Hareesh

Kailas Jith V A

Sudeep S S

ABSTRACT

Travel Pro is a modern and efficient software solution designed to streamline, automate, and simplify the entire travel planning and management process through a secure and user friendly web based digital platform. The system is divided into three primary modules Admin User and Service Providers Hotels and Cars each with distinct roles to ensure smooth and comprehensive management of travel services. Administrators can manage destinations hotels and vehicles oversee bookings review user itineraries handle service provider operations and respond to feedback or complaints in real time ensuring operational efficiency and data integrity. Users can register log in explore destinations search and filter hotels or vehicles based on preferences view service details book itineraries and submit feedback or complaints all through an intuitive interface. Service providers can manage their offerings update availability confirm bookings and respond to user feedback efficiently. The platform focuses on real time responsiveness intuitive design and reliable performance ensuring that administrators users and service providers experience minimal complexity in interaction. By digitizing the traditional travel workflow Travel Pro reduces manual tasks enhances operational control and elevates user satisfaction through improved transparency accessibility and convenience. In essence Travel Pro is a complete digital travel management ecosystem that transforms the way travel planning and bookings are handled promoting efficiency convenience and engagement through thoughtful design and smart technology

CONTENTS

1. INTRODUCTION	1
1.1 ABOUT THE PROJECT	2
1.2 ORGANISATIONAL OVERVIEW	3
2. SYSTEM ANALYSIS	4
2.1 EXISTING SYSTEM	6
2.1.1 Limitations of Existing System	6
2.2 PROPOSED SYSTEM	7
2.2.1 Advantages of Proposed System	7
2.2.2 Features of Proposed System	7
2.3 REQUIREMENTS SPECIFICATIONS	8
2.3.1 Hardware Requirements	8
2.3.2 Software Requirements	8
2.3.3 Functional Requirements	9
2.3.4 Non-Functional Requirements	15
2.4 FEASIBILITY STUDY	16
2.4.1 Economic Feasibility	16
2.4.2 Operational Feasibility	17
2.4.3 Technical Feasibility	17
3. SYSTEM DESIGN	18
3.1 DESIGN OF SUBSYSTEM.....	19
3.2 USER INTERFACE DESIGN	21
3.2.1 Input Design	21
3.2.2 Output Design	22
3.3 DATABASE DESIGN	23
3.3.1 Table Design	26

3.3.2 Data Dictionary	33
3.4 MODELLING	35
3.4.1 Data flow Diagram (DFD)	36
3.4.2 Entity Relationship Model	42
3.5 ARCHITECTURAL DESIGN	44
3.5.1 Structure Chart	44
3.6 PROCEDURAL DESIGN	45
3.6.1 Flow Chart	45
4. PROCESSING ENVIRONMENT	52
4.1 HARDWARE SPECIFICATIONS	53
4.2 SOFTWARE SPECIFICATIONS	55
5. CODING AND IMPLEMENTATION	56
5.1 CODING	57
5.2 TESTING	69
5.2.1 Unit Testing	69
5.2.2 Integration Testing	70
5.2.3 System Testing	71
6. SECURITY, BACKUP AND RECOVERY MECHANISM	72
7. FUTURE ENHANCEMENT	74
8. SOFTWARE MAINTENANCE	76
9.CONCLUSION	79
10. APPENDIX	80
GANTT CHART.....	81
SCREENSHOTS	82
MENU TREE	88
MEETING MINUTES.....	89
11. BIBLIOGRAPHY	93

INTRODUCTION

1.1 ABOUT THE PROJECT

Our project, Travel Pro, is a smart and user-centric travel management platform designed to provide a seamless, efficient, and accessible solution for planning and managing travel activities online. Developed with both travellers and service providers in mind, the system offers an intuitive interface that simplifies itinerary creation, hotel booking, and car reservation processes. Users can easily explore destinations, view available hotels and cars, check real-time availability, and receive instant booking confirmations. Each listing provides detailed information including pricing, availability status, location details, and supporting images, enabling users to make well-informed travel decisions. The platform is accessible 24/7 and supports multiple devices such as smartphones, tablets, and desktops, ensuring travellers can plan and manage their trips anytime and anywhere with ease.

By digitizing and centralizing the traditional travel planning process, Travel Pro eliminates the need to rely on multiple disconnected platforms and manual coordination. This approach not only saves time for users but also enhances operational efficiency for administrators and service providers. The platform supports essential administrative functionalities such as destination management, hotel and car service management, booking record maintenance, and content moderation. Additionally, interactive features like travel blogs and shared itineraries help increase user engagement and community participation. In summary, Travel Pro delivers a reliable, efficient, and modern travel management solution that prioritizes convenience, organization, and an enhanced user experience.

1.2 ORGANISATIONAL OVERVIEW

Ainsoft Solutions Pvt. Ltd. is a leading IT service provider offering comprehensive technology solutions across India and the Middle East, including countries such as the UAE, Saudi Arabia, and Kuwait. The company has established itself as a complete IT solution provider in the field of information technology, delivering high-quality services to a wide range of clients globally. Ainsoft Solutions is powered by a young, creative, and talented team of professionals, including developers, designers, and digital marketing analysts, who work collaboratively to address diverse business needs. The organization specializes in multiple domains such as software development, web design and development, mobile application development, graphic design, and digital marketing services. With a strong focus on innovation and customer satisfaction, the company strives to create solutions that maximize value for clients while enhancing their business performance and operational efficiency. Each project is approached with a thorough understanding of client requirements and is executed with precision to achieve optimal results. Through its commitment to excellence and continuous improvement, Ainsoft Solutions aims to deliver reliable, efficient, and scalable IT solutions that support business growth and digital transformation.

SYSTEM ANALYSIS

INTRODUCTION TO SYSTEM ANALYSIS

System analysis is a critical and foundational phase of the Software Development Life Cycle (SDLC), in which the architecture, behaviour, and detailed requirements of the proposed system are systematically examined to ensure successful development and implementation. In the case of the Travel Pro travel management platform, this phase begins with a comprehensive evaluation of the system's functional expectations, including user registration, secure authentication, destination selection, itinerary creation, hotel booking, car reservation, and the ability to share travel blogs and experiences. On the administrative side, key operations such as destination management, hotel and car listing approval, booking supervision, service provider management, and user content moderation are identified as core functionalities. The analysis phase also involves collecting relevant data about existing fragmented travel planning methods, studying user interaction patterns across multiple platforms, and identifying common inefficiencies in traditional travel management systems. This process helps establish a clear understanding of the challenges the proposed solution must address and improve.

During system analysis, logical models and process flow diagrams are prepared to visually represent how information moves within the platform, such as a travel booking request transitioning from the user interface to the administrative dashboard for verification and confirmation. Special attention is given to real-time components, including live hotel and car availability updates, instant booking confirmations, and timely administrative responses to user-generated content or service-related issues. These elements are carefully planned to enhance reliability, transparency, and overall user satisfaction. By conducting a detailed and structured system analysis, the Travel Pro project lays a strong foundation for the subsequent design and development phases, ensuring that technical planning aligns closely with practical usability and that the final system effectively meets both user expectations and operational requirements.

2.1 EXISTING SYSTEM

In the present system, many travellers face difficulties while planning and managing their trips due to scattered information, unreliable sources, and inefficient service coordination. Travelers expect a solution that is quick, secure, and convenient, but existing travel planning systems often fail to meet these expectations. Issues such as unorganized itinerary planning, delayed booking confirmations, and lack of real-time updates for hotels and transportation services are common. Especially during peak travel seasons or in popular destinations, users struggle to find reliable accommodation and transport options that suit their preferences and budgets. Moreover, traditional travel planning methods usually involve manual coordination across multiple platforms and unclear booking policies, leading to confusion and dissatisfaction. The existing system lacks proper integration and requires excessive time and effort to plan and manage travel smoothly.

2.1.1 LIMITATION OF EXISTING SYSTEM

- Time-consuming to access complete travel information across multiple platforms
- Often provides incorrect or outdated details regarding hotels, transportation, and itineraries
- Lack of proper security measures and reliable user verification mechanisms
- Limited availability of travel services based on destination and season
- Hidden or unclear charges related to bookings and service fees
- Limited hotel and transportation options with no efficient filtering or sorting features

2.2 PROPOSED SYSTEM

In the proposed system, users can plan and manage their travel activities at any time through a secure and user-friendly online platform. This system eliminates the delays and confusion present in traditional travel planning methods by offering real-time availability, instant booking confirmations, and intelligent filtering options for hotels, cars, and travel destinations. The Travel Pro platform is fully automated and designed for efficient interaction, requiring only minimal steps to complete itinerary creation and bookings. Both users and administrators have separate login panels with authorized access. Users can easily explore destinations, book hotels and cars based on their preferences, and share travel experiences, while administrators can manage bookings, monitor user activity, and control service availability. This web-based application provides a reliable and efficient alternative to fragmented travel systems, reducing manual effort while significantly enhancing overall user satisfaction.

2.2.1 ADVANTAGES OF PROPOSED SYSTEM

- No paperwork or manual coordination across multiple travel platforms
- Real-time access to available hotels, cars, and travel services
- Instant booking confirmations for hotels and transportation
- Travel blogs, feedback, and support options
- User and admin login system with authorized access
- Convenient, fast, and secure travel planning and booking process

2.2.2 Features of Proposed System

- Users can register, log in, explore destinations, and filter hotels and cars based on preferences
- Hotel and car bookings can be made instantly and tracked within the system
- Administrators can approve bookings, manage service availability, and respond to user feedback
- The system verifies login credentials to ensure secure and authenticated user access
- Pending and approved bookings are displayed separately for better clarity
- Maintains complete records of bookings, service status, and user interactions

2.3 REQUIREMENT SPECIFICATIONS

2.3.1 HARDWARE REQUIREMENTS

Processor	:	Intel core i7 Processor
Speed	:	3GHz or above
RAM	:	3GB or above
Hard Disk Capacity	:	1TB
Keyboard	:	Multimedia
Keyboard Mouse	:	Standard
USB	:	2.0 & 3.0

2.3.2 SOFTWARE REQUIREMENTS

Operating System	:	Windows 11
Front End	:	React.js
Backend	:	ASP.NET WEB API(C#)
Language	:	C#(Backend),JavaScript (Frontend)
Tools used	:	Visual Studio ,Visual Studio Code
Database	:	SQL Server

2.2.3 Functional Requirements

A Functional Requirement (FR) defines the specific services and operations that the Travel Pro platform must provide. It describes the expected behaviour, interactions, and outputs of the system and its individual modules. These requirements cover areas such as data handling, user and administrator roles, interface interactions, and core travel management functionalities including itinerary planning, hotel booking, and car reservations. Functional Requirements serve as a foundation for system development and ensure that responsibilities of both users and administrators are clearly defined. These requirements are also commonly referred to as Functional Specifications.

Admin Module

The admin module enables authorized administrators to log in and manage the core operations of the Travel Pro platform. Administrators can securely access the system to manage destinations, hotel listings, and car service details by adding, updating, or removing records as required. They are responsible for verifying and managing user accounts, approving or moderating itineraries, and overseeing bookings made by users. Administrators can also review and respond to user feedback, blogs, and complaints, as well as access complete booking and activity records across the platform. This module ensures effective control over service management, data accuracy, and overall system integrity.

FN 1: View Customers

Allows admin to view customer details stored in the system

Input: Customer ID

Output: Customer Details Display

FN 2: View Journeys

Allows admin to view customers journeys

Input: Journey ID

Output: Journey Details Display

FN 3: Manage Drivers

Allows admin to add, update, or delete driver information

Input: Driver Name

Output: Driver Details Updated Successfully

FN 4: View Cab Booking

Allows admin to view cab booking records

Input: Booking ID

Output: Cab Booking Details Displayed

FN 5: Manage Destination

Allows admin to add or update destination details

Input: Destination Name

Output: Destination Details Updated Successfully

FN 6: Manage Hotel

Allows admin to manage hotel information

Input: Hotel Name

Output: Hotel Details Updated Successfully

FN 7: View Hotel Booking

Allows admin to view hotel booking records

Input: Booking ID

Output: Hotel Booking Details Displayed

FN 8: Approve User

Allows admin to approve registered users in the system

Input: User ID

Output: User Approved Successfully

Customer Module

The User Module enables registered users to interact with the Travel Pro travel management platform. Users can initially register by providing their basic details and required information for verification. After successful authentication, users can log in using their credentials to access platform features. Once logged in, users can explore travel destinations, search and filter available hotels and car services based on preferences such as location, price, or service type, and view detailed service information. Users can create and manage travel itineraries, book hotels and cars by selecting travel dates and locations, and receive instant booking confirmations. Additionally, users can share travel experiences through blogs, submit feedback or complaints, and view their complete booking history from the dashboard for future reference. This module ensures a smooth, organized, and user-friendly travel planning and booking experience.

FN 1: Manage Journeys

Allows the user to create and manage travel journeys

Input: Journeys Name

Output: Journey Created / Updated Successfully

FN 2: Edit Profile

Allows the user to update personal account information

Input: Profile ID

Output: Profile Updated Successfully

FN 3: View Cabs

Allows the user to view available cab services

Input: Cab ID

Output: List of Available Cabs Displayed

FN 4: Manage Cab Booking

Allows the user to book or manage cab services

Input: Booking ID

Output: Cab Booking Confirmed / Updated Successfully

FN 5: View Hotels

Allows the user to view available hotels

Input: Hotel ID

Output: List of Hotels Displayed

FN 6: Manage Hotel Booking

Allows the user to book and manage hotel reservations

Input: Booking ID

Output: Hotel Booking Confirmed / Updated Successfully

FN 7: View Destination

Allows the user to view travel destinations

Input: Destination ID

Output: Destination Details Displayed

Driver Module

The Driver Module allows registered drivers to manage their cab details and view booking information within the Travel Pro system. Drivers can update their profile, manage cab availability, and access booking details assigned to them. This module ensures smooth coordination between drivers and the system for efficient transportation services.

FN 1: Manage Cabs

Allows the driver to manage cab details and availability

Input: Cab Name

Output: Cab Details Updated Successfully

FN 2: Edit Profile

Allows the driver to update personal account information

Input: Profile ID

Output: Profile Updated Successfully

FN 3: View Cab Bookings

Allows the driver to view cab booking details

Input: Booking ID

Output: Cab Booking Details Displayed

Agency Module

The Agency Module allows registered car service agencies to manage cab details, drivers, and booking information within the Travel Pro system. Agencies can update cab information, maintain driver records, and view booking details for efficient transportation service management. This module ensures smooth coordination between cabs, drivers, and customer bookings.

FN 1: Manage Cabs

Allows the agency to add or update cab details

Input: Cab ID, Name, Price, Model, Year, Status

Output: Cab Details Added / Updated Successfully

FN 2: Manage Drivers

Allows the agency to add or update driver information

Input: Driver ID, Name, Contact Details, License Information

Output: Driver Details Added / Updated Successfully

FN 3: View Cab Booking

Allows the agency to view cab booking details

Input: Booking ID, Driver Details, Customer Details

Output: Cab Booking Details Displayed

Hotel Module

The Hotel Service Provider Module allows registered hotel providers to manage their hotel listings on the Travel Pro platform. Providers can securely log in and add new hotels, update room availability, pricing, and amenities, or remove outdated listings. They can upload images and descriptions to ensure travellers have accurate information for booking. Providers can view booking requests, confirm reservations, track upcoming and past bookings, and respond to user feedback. This module ensures smooth management of hotel operations and real-time updates for users.

FN 1: Manage Rooms

Allows the hotel provider to add or update room details

Input: Room Name

Output: Room Details Added / Updated Successfully

FN 2: Manage Highlights

Allows the hotel provider to manage hotel highlights and features

Input: Highlight Name

Output: Highlights Updated Successfully

FN 3: View Hotel Booking

Allows the hotel provider to view booking details of rooms

Input: Booking ID

Output: Hotel Booking Details Displayed

2.3.4 Non-Functional Requirements

In systems, a non-functional requirement (NFR) is a requirement that specifies criteria that can be used to judge the operation of a system, rather than specific behaviours. They are contrasted with functional requirements that define specific behaviour or functions:

- **Usability:** Defines how easily a user can learn and operate the Travel Pro system. It measures the efficiency of use, including the average time required to complete travel-related tasks, the number of tasks completed without assistance, and the ability to complete transactions without errors.
- **Authenticity:** The system ensures authenticity by allowing access only to users with valid usernames and passwords, thereby preventing unauthorized usage of the platform.
- **Data Integrity:** Data integrity is maintained by ensuring that all stored information related to users, bookings, itineraries, hotels, and car services remains accurate, consistent, and reliable.
- **Performance:** The system ensures good performance by maintaining low response time, high throughput, and efficient resource utilization during travel searches, bookings, and updates.
- **Availability:** The Travel Pro application is available at all times, allowing users to access travel services and manage bookings without interruption.
- **Security:** All data is securely stored in the database, and access is restricted to authorized users through authentication and role-based access control mechanisms.

2.4 FEASIBILITY STUDY

Feasibility study is the test of the system proposal made to identify whether the user needs may be satisfied using the current software and hardware technologies, whether the system will be cost effective from a business point of view and whether it can be developed with the given budgetary constraints. A feasibility study should be relatively cheap and done at the earliest possible time. Depending on the study, the decision is made whether to go ahead with a more detailed analysis. When a new project is proposed, it normally goes through feasibility assessment. Feasibility study is carried out to determine whether the proposed system is possible to develop with available resources and what should be the cost consideration. Facts considered in the feasibility analysis were:

- Economic Feasibility
- Operational Feasibility

2.4.1 Economic feasibility

This feasibility study present tangible and intangible benefits from the prefect by comparing the development and operational cost. This system needs some more initial investment than the existing system, but it can be justifiable that it will improve quality of service. Thus, feasibility study should centre along the following points:

- Improvement resulting over the existing method in terms of accuracy, timeliness.
- Cost comparison
- Estimate on the life expectancy of the hardware
- Overall objective

Our project is economically feasible. It does not require much cost to be involved in the overall process. It is essential because the main goal of the proposed system is to have economically better result with the increase.

2.4.2 Operational Feasibility

Operational analysis is the most frequently used method for evaluating the effectiveness of a new system. More commonly known as cost/benefit analysis, the procedure is to determine the benefits and saving that are expected from a candidate system and compare them with costs. If benefits outweigh costs, then the decision is made to design and implement the system. An entrepreneur must accurately weigh the cost versus benefits before taking an action. Cost based study: It is important to identify cost and benefit factors, which can be categorized as follows:

1. Development costs.
2. Operating costs.

This is an analysis of the costs to be incurred in the system and benefits derivable out of the system. Time-based study: This is an analysis of the time required to achieve a return on investments. The future value of a project is also a factor. This application can be operated by all the users across the world.

2.4.3 Technical feasibility

Technical Feasibility deals with the hardware as well as software requirements. Technology is not a constraint to type system development. We have to find out whether the necessary technology, the proposed equipment has the capacity to hold the data, which is used in the project, should be checked to carry out this technical feasibility. The technical feasibility issues usually raised during the feasibility stage of investigation includes:

- This software is running in Windows 11.
- The minimum hardware required is 3GB and maximum hardware required is 400GB.
- The system can be expanded.

SYSTEM DESIGNS

3.1 DESIGN OF SUBSYSTEM

A Design of Subsystem encapsulates behaviour, providing explicit and formal interfaces while hiding internal implementation details. This design approach helps organize and manage the interactions between multiple classes and subsystems. The Travel Pro platform is structured around three core modules that support the complete functioning of the system

1. ADMIN MODULE

The Admin Module is responsible for managing and monitoring the overall Travel Pro platform. Administrators oversee customer activities, journey records, bookings, and service-related data such as vehicles, destinations, and hotels. They also ensure system integrity by approving users and maintaining accurate records across all modules.

Main functionalities include:

- Admin can view customers and access their registered details within the system.
- Admin can view journeys to monitor user travel plans and activities.
- Admin can manage drivers by maintaining and updating driver-related information.
- Admin can view cab bookings to track transportation service usage.
- Admin can manage destinations by adding or updating travel location details.
- Admin can manage hotels by maintaining hotel-related information.
- Admin can view hotel bookings to monitor accommodation reservations.
- Admin can approve users to allow access to the platform.

2. CUSTOMER MODULE

The Customer Module enables users to interact with the Travel Pro system by planning and managing their travel activities. Users can organize journeys, explore available services, and handle bookings efficiently through a user-friendly interface.

Main functionalities include:

- User can manage journeys by creating and updating travel plans.
- User can edit profile to keep personal information up to date.
- User can view cabs to explore available transportation options.
- User can manage cab booking by booking or modifying cab services.
- User can view hotels to check available accommodation options.
- User can manage hotel booking by reserving or updating hotel stays.
- User can view destination to explore travel locations and details.

3. DRIVER MODULE

The Driver Module allows drivers to manage their assigned services and maintain their information within the system. It ensures drivers can efficiently handle transportation responsibilities and stay updated with booking details.

Main functionalities include:

- Driver can manage cabs by updating cab details and availability status.
- Driver can edit profile to maintain accurate personal and contact information.
- Driver can view cab bookings to access assigned trip and customer details.

4. AGENCY MODULE

The Agency Module enables agencies to manage transportation resources, including cabs and drivers, while also keeping track of booking activities. It helps ensure proper coordination between vehicles, drivers, and customer bookings.

Main functionalities include:

- Agency can manage cabs by adding or updating cab information and availability.
- Agency can manage drivers by maintaining driver records and details.
- Agency can view cab booking to monitor customer bookings and trip details.

5. HOTEL MODULE

The Hotel Module allows hotel providers to manage room information, highlights, and booking records within the system. It ensures efficient handling of accommodation services and accurate booking management.

Main functionalities include:

- Hotel provider can manage rooms by adding or updating room details and pricing.
- Hotel provider can manage highlights by maintaining hotel features and descriptions.
- Hotel provider can view hotel booking to access reservation details and customer information

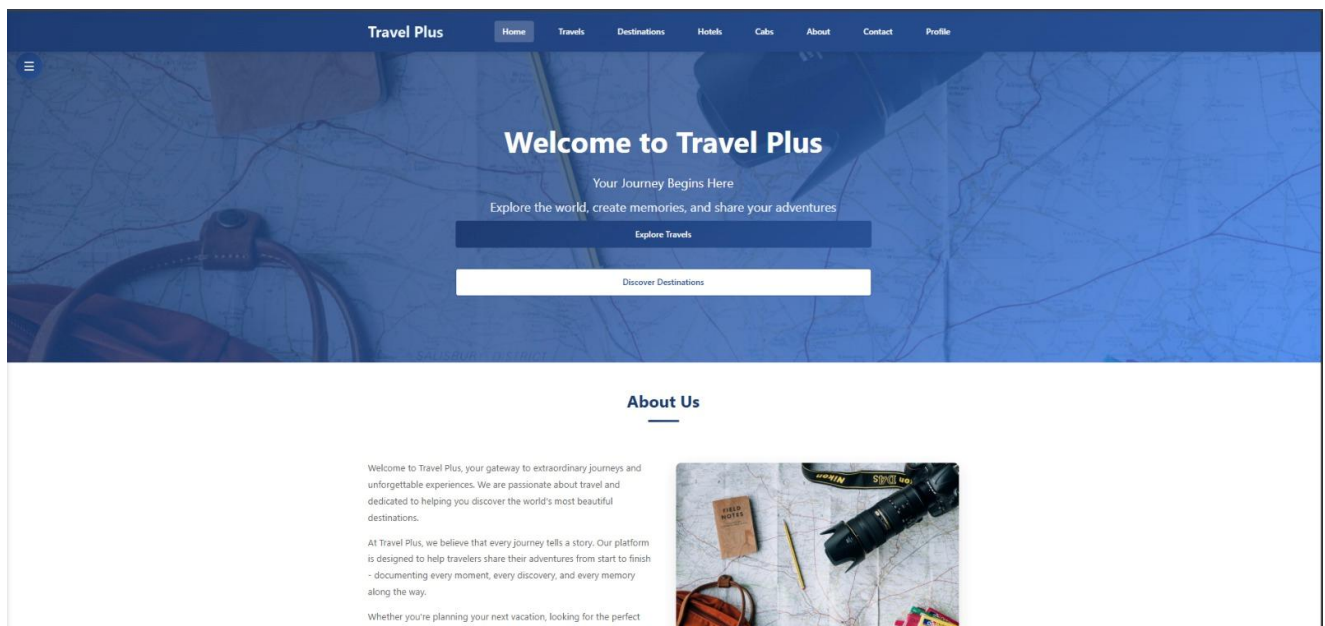
.

3.2 USER INTERFACE DESIGN

3.2.1 Input Design

Input designing is the basic theory to be considered during system study. The input media used in the system is the keyboard. Details are entered in the system through different data entry screens. The system is designed in a user-friendly manner. Appropriate error messages are displayed when a false data is entered. The user interface design is very important for any application. The interface design defines how the software communicates within itself, to system that interpreted with it and with human who use it. The input design is the process of converting the user-oriented description of inputs into a programmer-oriented specification. It is the link that ties the system into the world of its users. Input design involves determining the record media, method of input, speed of capture and entry to the system. The analyst should consider the following points when designing the input:

User interface-input



3.2.2 Output Design

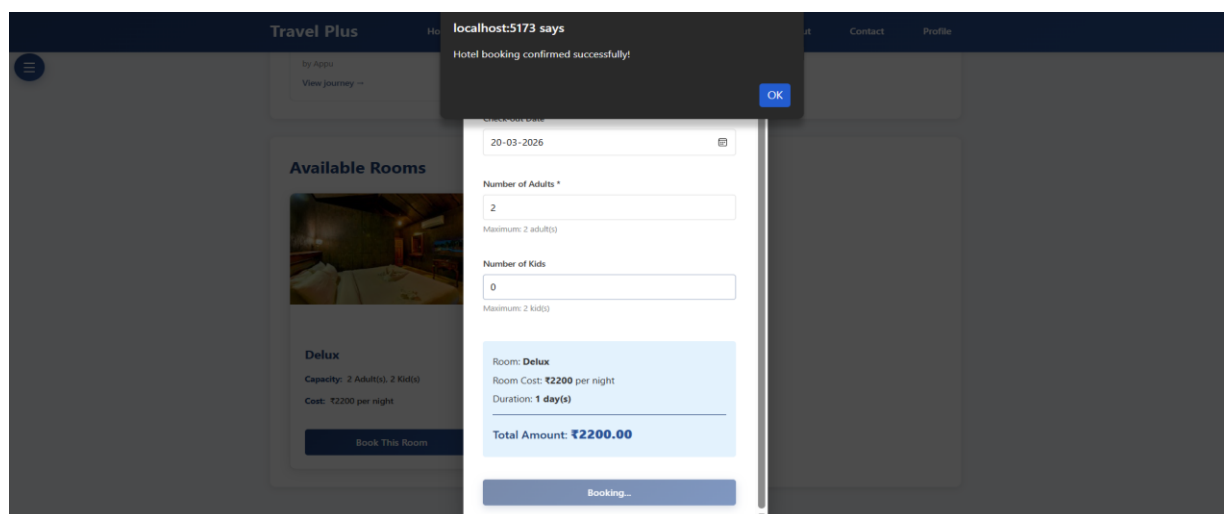
An inevitable activity in the system design is the proper design of output in a form acceptable to the user. Outputs from the system are required primarily to communicate the result of processing to users. Outputs also provide a permanent copy of the results for later consultation. An intelligible output will improve system relationship with the user and help in the decision making process.

The approach to output design is very dependent on the type of output and nature of data. Special attention has to be made to data editing. The choice of appropriate output medium is also an important task. The selection may be affected by the following kinds of consideration:

- Response time.
- Location of users.
- Cost.
- Software/hardware.
- Suitability of the device for application concerned.

The output design must be specified and documented, data items have to be accurately defined and arranged for clarity and easy comprehension.

User interface- output



3.3 DATABASE DESIGN

The overall objective in the development of database technology has been to treat data as an organizational resource and as an integrated whole. DBMS allow data to be protected and organized separately from other resources. Database is an integrated collection of data. The most significant form of data as seen by the programmers is data as stored on the direct access storage devices. This is the difference between logical and physical data.

Database files are the key source of information into the system. It is the process of designing database files, which are the key source of information to the system. The files should be properly designed and planned for collection, accumulation, editing and retrieving the required information. The organization of data in database aims to achieve three major objectives:

- Data integration.
- Data integrity.
- Data independence.

The proposed system stores the information necessary for processing in the MySQL database. MySQL is a widely-used open-source relational database management system (RDBMS). Unlike traditional NoSQL databases, MySQL employs a structured query language (SQL) for data manipulation and retrieval. It is renowned for its reliability, ease of use, and robust transaction support, making it ideal for various applications. In MySQL, data consistency is maintained through a multi-master replication architecture. This setup ensures that multiple nodes can accept write operations, enhancing scalability and fault tolerance. Updates to the primary database are automatically propagated to secondary databases, ensuring redundancy and seamless failover capabilities. In terms of data organization, MySQL uses tables composed of rows and columns. Each row represents a record, containing fields that store specific data types such as integers, strings, and dates. Relationships between tables are established through primary and foreign key constraints, ensuring data integrity and facilitating complex queries. Overall, MySQL's relational model and SQL based interface offer a robust foundation for managing structured data efficiently.

Normalization

Normalization is a technique of separating redundant fields and breaking up a large table into a smaller one. It is also used to avoid insertion, deletion and updating anomalies. The table has been normalized up to the third normal form. It is the process of structuring a relational database in accordance with a series of so-called normal forms in order to reduce data redundancy and improve data integrity. Normalization entails organizing the columns (attributes) and tables (relations) of a database to ensure that their dependencies are properly enforced by database integrity constraints. It is accomplished by applying some formal rules either by a process of synthesis (creating a new database design) or decomposition (improving an existing database design). In short, the rules for each of the five normal forms are as follows:

- First normal form

A relation is said to be in 1NF if all the underlying domain of attributes contain simple individual values.

- Second normal form

The 2NF is based on the concept of full functional dependency. A relation is said to be in 2NF if and only if it is in 1NF and every non-key attribute is fully functionally dependent on candidate key of the table.

- Third normal form

The 3NF is based on the concept of transitive dependency. A relation in 2NF is said to be in 3NF if every non-key attribute is non-transitively dependent on candidate key of the table. Our project is in the second normal form because every non key attribute is fully functionally dependent on primary key. A relation is in third normal form, if there is no transitive dependency for nonprime attributes as well as it is in second normal form.

- o Boyce Codd normal form (BCNF)

BCNF is the advance version of 3NF. It is stricter than 3NF. A table is in BCNF if every functional dependency $X \rightarrow Y$, X is the super key of the table. For BCNF, the table should be in

3NF, and for every FD, LHS is super key

- Fourth normal form (4NF)

A relation will be in 4NF if it is in Boyce Codd normal form and has no multi-valued dependency. For a dependency $A \twoheadrightarrow B$, if for a single value of A, multiple values of B exist, then the relation will be a multi-valued dependency.

- Fifth normal form (5NF)

A relation is in 5NF if it is in 4NF and not contains any join dependency and joining should be lossless. 5NF is satisfied when all the tables are broken into as many tables as possible in order to avoid redundancy. 5NF is also known as Project-join normal form (PJ/NF). Our project Travel pro is based on second normal form (2NF).

3.3.1 Table Design

Database design activity deals with the structure of the physical database. A key objective is to define how access paths are implemented. A physical path is derived from a logical structure through keys, pointers, and chains. Below are the main tables used in this project:

Users

Field Name	Data Type	Constraints
Id	BIGINT	PRIMARY KEY
UserName	NVARCHAR(150)	NOT NULL
Password	NVARCHAR(200)	NOT NULL
PasswordSalt	NVARCHAR(300)	NULL
Role	NVARCHAR(100)	NOT NULL
IsBlocked	BIT	NULL
CustomerId	BIGINT	FOREIGN KEY
DriverId	BIGINT	FOREIGN KEY
HotelId	BIGINT	FOREIGN KEY
IsVerified	BIT	NULL

Customers

Field Name	Data Type	Constraints
Id	BIGINT	PRIMARY KEY
Name	NVARCHAR(100)	NOT NULL
MobileNo	NVARCHAR(10)	NOT NULL
EmailId	NVARCHAR(100)	NULL
Location	NVARCHAR(100)	NULL
Address	NVARCHAR(200)	NULL
IsActive	BIT	NULL

Migration History

Field Name	Data Type	Constraints
MigrationId	NVARCHAR(150)	PRIMARY KEY
ContextKey	NVARCHAR(300)	NOT NULL
Model	VARBINARY(MAX)	NOT NULL
ProductVersion	NVARCHAR(32)	NOT NULL

Brands

Field Name	Data Type	Constraints
Id	BIGINT	PRIMARY KEY
Name	NVARCHAR(100)	NULL
IsActive	BIT	NULL
Photo	NVARCHAR(200)	NULL

Destinations

Field Name	Data Type	Constraints
Id	BIGINT	PRIMARY KEY
Name	NVARCHAR(100)	NOT NULL
Description	VARCHAR(500)	NULL
Photo	VARCHAR(200)	NULL
IsActive	BIT	NULL
CountryId	BIGINT	FOREIGN KEY

Countries

Field Name	Data Type	Constraints
Id	BIGINT	PRIMARY KEY
Name	NVARCHAR(100)	NOT NULL
IsActive	BIT	NULL

Cab Bookings

Field Name	Data Type	Constraints
Id	BIGINT	PRIMARY KEY
CabId	BIGINT	NOT NULL
PickPlace	VARCHAR(100)	NOT NULL
Time	NVARCHAR(10)	NOT NULL
TotalKm	INT	NOT NULL
TotalAmount	REAL	NOT NULL
Rating	INT	NULL
Status	NVARCHAR(100)	NULL
BookedDate	DATETIME	NOT NULL
CustomerId	BIGINT	FOREIGN KEY
DriverId	BIGINT	FOREIGN KEY
VehicleId	BIGINT	FOREIGN KEY
DestinationId	BIGINT	FOREIGN KEY

Vehicle Types

Field Name	Data Type	Constraints
Id	BIGINT	PRIMARY KEY
Name	NVARCHAR(100)	NOT NULL
IsActive	BIT	NULL

Journeys

Field Name	Data Type	Constraints
Id	BIGINT	PRIMARY KEY
Photo	NVARCHAR(200)	NULL
DateFrom	DATETIME	NOT NULL
DateTo	DATETIME	NOT NULL
NoOfDays	INT	NOT NULL
Title	NVARCHAR(200)	NOT NULL
Description	NVARCHAR(1000)	NULL
CustomerId	BIGINT	FOREIGN KEY
IsActive	BIT	NULL

User Sessions

Field Name	Data Type	Constraints
Id	BIGINT	PRIMARY KEY
Token	NVARCHAR(256)	NOT NULL
SessionTimeStamp	DATETIME	NOT NULL
ExpiresInMinutes	BIGINT	NOT NULL
UserId	BIGINT	FOREIGN KEY
FinancialYearId	BIGINT	NULL
UserSessionStatus	INT	NULL

Journey Hotels

Field Name	Data Type	Constraints
Id	BIGINT	PRIMARY KEY
HotelId	BIGINT	FOREIGN KEY
DestinationId	BIGINT	FOREIGN KEY
JourneyDetailId	BIGINT	FOREIGN KEY
JourneyId	BIGINT	FOREIGN KEY
IsActive	BIT	NULL

Vehicles

Field Name	Data Type	Constraints
Id	BIGINT	PRIMARY KEY
Model	NVARCHAR(100)	NOT NULL
Year	INT	NOT NULL
FuelType	NVARCHAR(100)	NOT NULL
Transmission	NVARCHAR(100)	NOT NULL
Color	NVARCHAR(100)	NOT NULL
NoOfSeat	INT	NOT NULL
Rate	DECIMAL(18,2)	NOT NULL
Image1	NVARCHAR(200)	NULL
Image2	NVARCHAR(200)	NULL
Image3	NVARCHAR(200)	NULL
Image4	NVARCHAR(200)	NULL
Image5	NVARCHAR(200)	NULL
Luggage	INT	NULL
Sensors	BIT	NULL
Bluetooth	BIT	NULL
Camera	BIT	NULL
LCD	BIT	NULL
Safety	BIT	NULL
BrandId	BIGINT	FOREIGN KEY
VehicleTypeId	BIGINT	FOREIGN KEY
DriverId	BIGINT	FOREIGN KEY

Journey Hotels (Details)

Field Name	Data Type	Constraints
Id	BIGINT	PRIMARY KEY
JourneyDetailId	BIGINT	FOREIGN KEY
HotelId	BIGINT	FOREIGN KEY
IsActive	BIT	NULL

Journey Galleries

Field Name	Data Type	Constraints
Id	BIGINT	PRIMARY KEY
JourneyId	BIGINT	FOREIGN KEY
Photo	NVARCHAR(200)	NULL

Journey Details

Field Name	Data Type	Constraints
Id	BIGINT	PRIMARY KEY
JourneyId	BIGINT	FOREIGN KEY
DestinationId	BIGINT	FOREIGN KEY
Title	NVARCHAR(200)	NOT NULL
Description	NVARCHAR(1000)	NULL
DayNumber	INT	NOT NULL
IsActive	BIT	NULL

Hotels

Field Name	Data Type	Constraints
Id	BIGINT	PRIMARY KEY
Name	NVARCHAR(100)	NOT NULL
Email	NVARCHAR(100)	NULL
MobileNo	NVARCHAR(100)	NULL
Description	NVARCHAR(MAX)	NULL
CostPerDay	DECIMAL(18,2)	NULL
Image1	NVARCHAR(200)	NULL
Image2	NVARCHAR(200)	NULL
Image3	NVARCHAR(200)	NULL
Image4	NVARCHAR(200)	NULL
Image5	NVARCHAR(200)	NULL
Location	NVARCHAR(200)	NULL
Address	NVARCHAR(200)	NULL
HotelTypeId	BIGINT	FOREIGN KEY
StarRating	INT	NULL
IsActive	BIT	NULL
RegistrationCertificate	NVARCHAR(200)	NULL
DestinationId	BIGINT	FOREIGN KEY

Hotel Highlights

Field Name	Data Type	Constraints
Id	BIGINT	PRIMARY KEY
HotelId	BIGINT	FOREIGN KEY
Name	NVARCHAR(500)	NOT NULL
IsActive	BIT	NULL

Hotel Rooms

Field Name	Data Type	Constraints
Id	BIGINT	PRIMARY KEY
HotelId	BIGINT	FOREIGN KEY
Title	NVARCHAR(100)	NOT NULL
Description	NVARCHAR(500)	NULL
Beds	INT	NULL
AC	INT	NULL
Rating	INT	NULL
Image1	NVARCHAR(200)	NULL
Image2	NVARCHAR(200)	NULL
Image3	NVARCHAR(200)	NULL
Image4	NVARCHAR(200)	NULL
Image5	NVARCHAR(200)	NULL
IsActive	BIT	NULL

Journey Destinations

Field Name	Data Type	Constraints
Id	BIGINT	PRIMARY KEY
JourneyId	BIGINT	FOREIGN KEY
DestinationId	BIGINT	FOREIGN KEY
IsActive	BIT	NULL

Hotel Bookings

Field Name	Data Type	Constraints
Id	BIGINT	PRIMARY KEY
Adult	INT	NULL
Kids	INT	NULL
TotalDays	INT	NULL
TotalPrice	REAL	NULL
FromDate	DATETIME	NULL
ToDate	DATETIME	NULL
HotelId	BIGINT	FOREIGN KEY
CustomerId	BIGINT	FOREIGN KEY
RoomId	BIGINT	FOREIGN KEY
BookingStatus	INT	NULL
IsActive	BIT	NULL

Hotel Types

Field Name	Data Type	Constraints
Id	BIGINT	PRIMARY KEY
Name	NVARCHAR(100)	NOT NULL
IsActive	BIT	NULL

Hotel Images

Field Name	Data Type	Constraints
Id	BIGINT	PRIMARY KEY
HotelId	BIGINT	FOREIGN KEY
Image	NVARCHAR(200)	NULL
IsActive	BIT	NULL

Hotel Reviews

Field Name	Data Type	Constraints
Id	BIGINT	PRIMARY KEY
HotelId	BIGINT	FOREIGN KEY
CustomerId	BIGINT	FOREIGN KEY
Rating	INT	NULL
Review	NVARCHAR(500)	NULL
IsActive	BIT	NULL

Hotel Amenities

Field Name	Data Type	Constraints
Id	BIGINT	PRIMARY KEY
HotelId	BIGINT	FOREIGN KEY
AmenityName	NVARCHAR(200)	NOT NULL
IsActive	BIT	NULL

3.3.2 Data Dictionary

Data Dictionary is the major component in the structured analysis model of the system. It lists all the data items appearing in DFD. A data dictionary is a file or a set of files that includes a database's metadata (hold records about other objects in the database), like data ownership, relationships of the data to another object, and some other data.

Field Name	Data Type	Constraints
admin_username	VARCHAR(50)	PRIMARY KEY
admin_password	VARCHAR(255)	NOT NULL
full_name	VARCHAR(100)	NOT NULL
email	VARCHAR(100)	UNIQUE, NOT NULL
password	VARCHAR(255)	NOT NULL
phone_number	VARCHAR(15)	NOT NULL
identity_proof	VARCHAR(100)	UNIQUE, NOT NULL
address	VARCHAR(255)	NOT NULL
username	VARCHAR(50)	NOT NULL
role	ENUM('user','admin')	DEFAULT 'user'
status	ENUM('pending','approved','rejected')	NOT NULL
cab_id	INT	PRIMARY KEY
cab_model	VARCHAR(100)	NOT NULL
fuel_type	ENUM('Petrol','Diesel','EV')	NOT NULL
vehicle_type	ENUM('Hatchback','Sedan','SUV')	NOT NULL
seating_capacity	INT	NOT NULL
price_per_km	DECIMAL(10,2)	NOT NULL
is_available	BOOLEAN	NOT NULL
image_url	VARCHAR(255)	NULL

booking_id	INT	PRIMARY KEY
trip_id	INT	UNIQUE
pickup_date	DATE	NOT NULL
pickup_time	TIME	NOT NULL
pickup_point	VARCHAR(255)	NULL
destination	VARCHAR(255)	NOT NULL
with_driver	BOOLEAN	NOT NULL
selected_cab	INT	NOT NULL
phone_no	VARCHAR(15)	NULL
booking_status	ENUM('pending','approved','rejected')	NOT NULL
admin_response	TEXT	NULL
complaint_id	INT	PRIMARY KEY
complaint_message	TEXT	NULL
related_trip_id	INT	NULL
driver_id	INT	PRIMARY KEY
drivename	VARCHAR(100)	NOT NULL
driver_phone	VARCHAR(15)	NOT NULL
driver_address	VARCHAR(255)	NOT NULL
driver_details	TEXT	NOT NULL
driver_age	INT	NOT NULL
license_number	VARCHAR(100)	NOT NULL
driver_status	ENUM('Available','Unavailable')	NOT NULL
rating	FLOAT	DEFAULT 0.0
history_id	INT	PRIMARY KEY

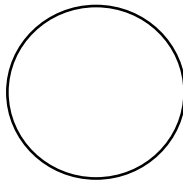
3.4 MODELLING

3.4.1 Data flow diagram (DFD)

A data flow diagram (DFD) is a graphical representation of the "flow" of data through an information system, modelling its process aspects. A DFD is often used as a preliminary step to create an overview of the system, which can later be elaborated. DFDs can also be used for the visualization of data processing (structured design). A DFD shows what kind of information will be input to and output from the system, where the data will come from and go to, and where the data will be stored. It does not show information about the timing of process or information about whether processes will operate in sequence or in parallel.

Data Flow Diagram Notations

Process/Function:



File/Database:



External Entities:

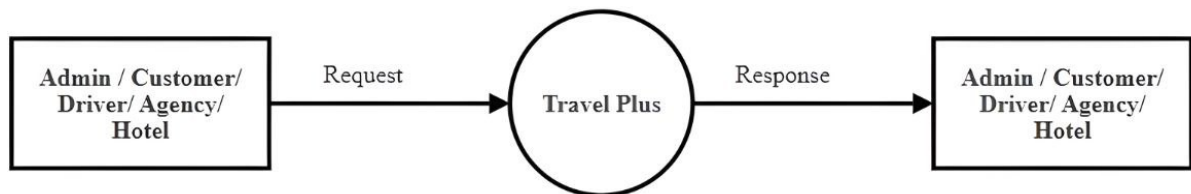


Data Flow:

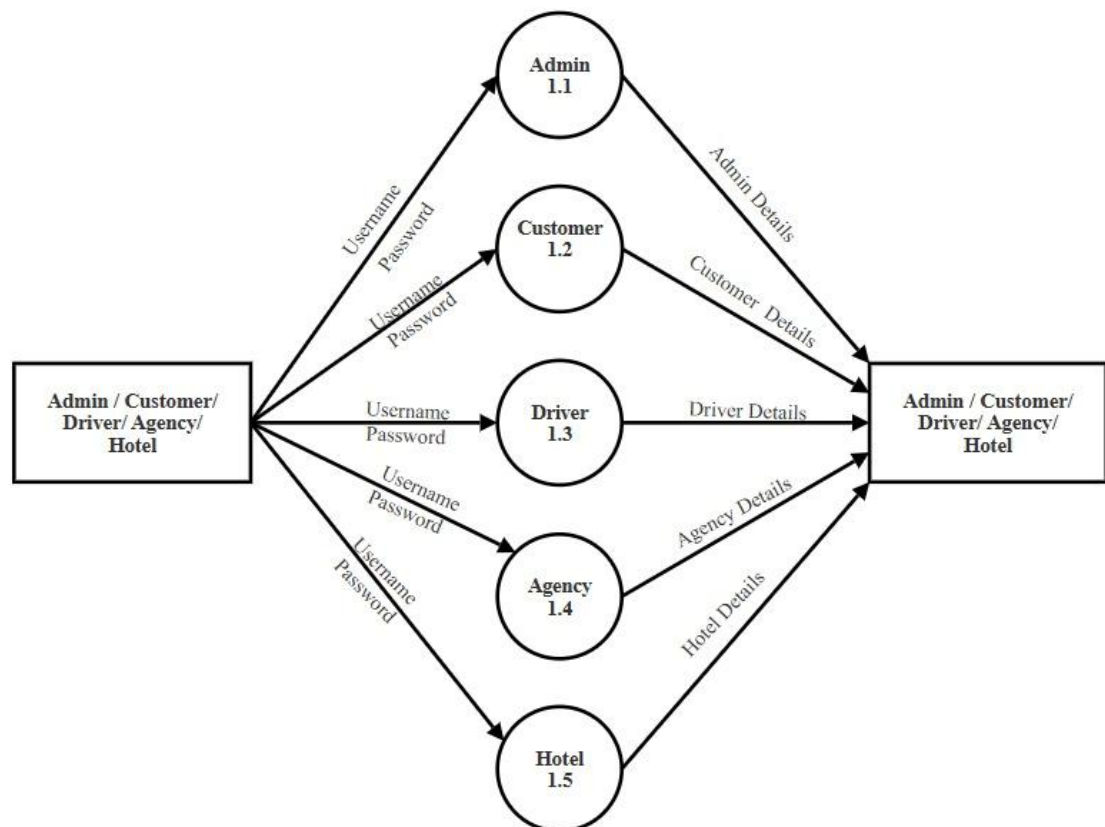


DATAFLOWDIAGRAM

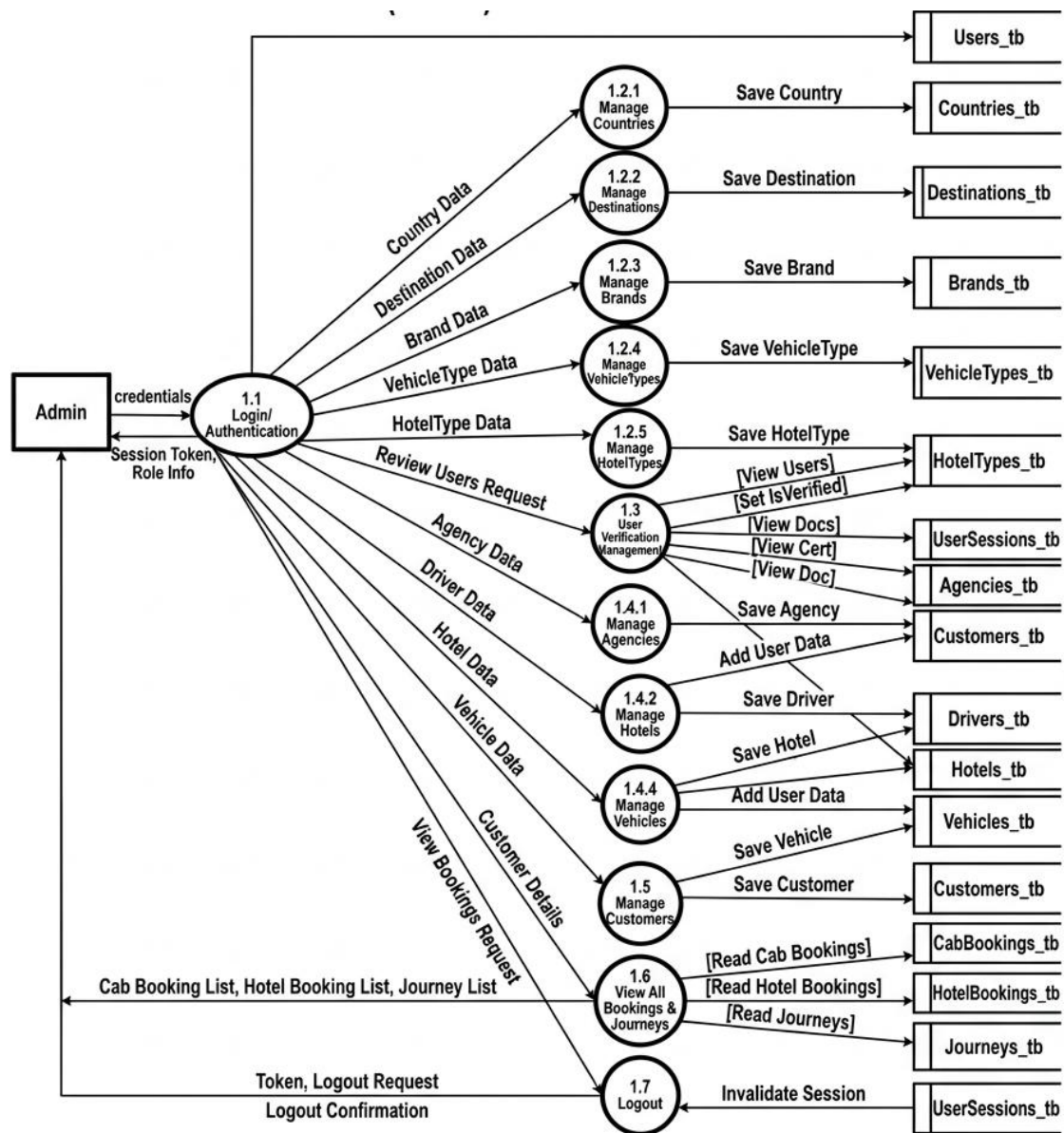
LEVEL-0/CONTEXTLEVEL



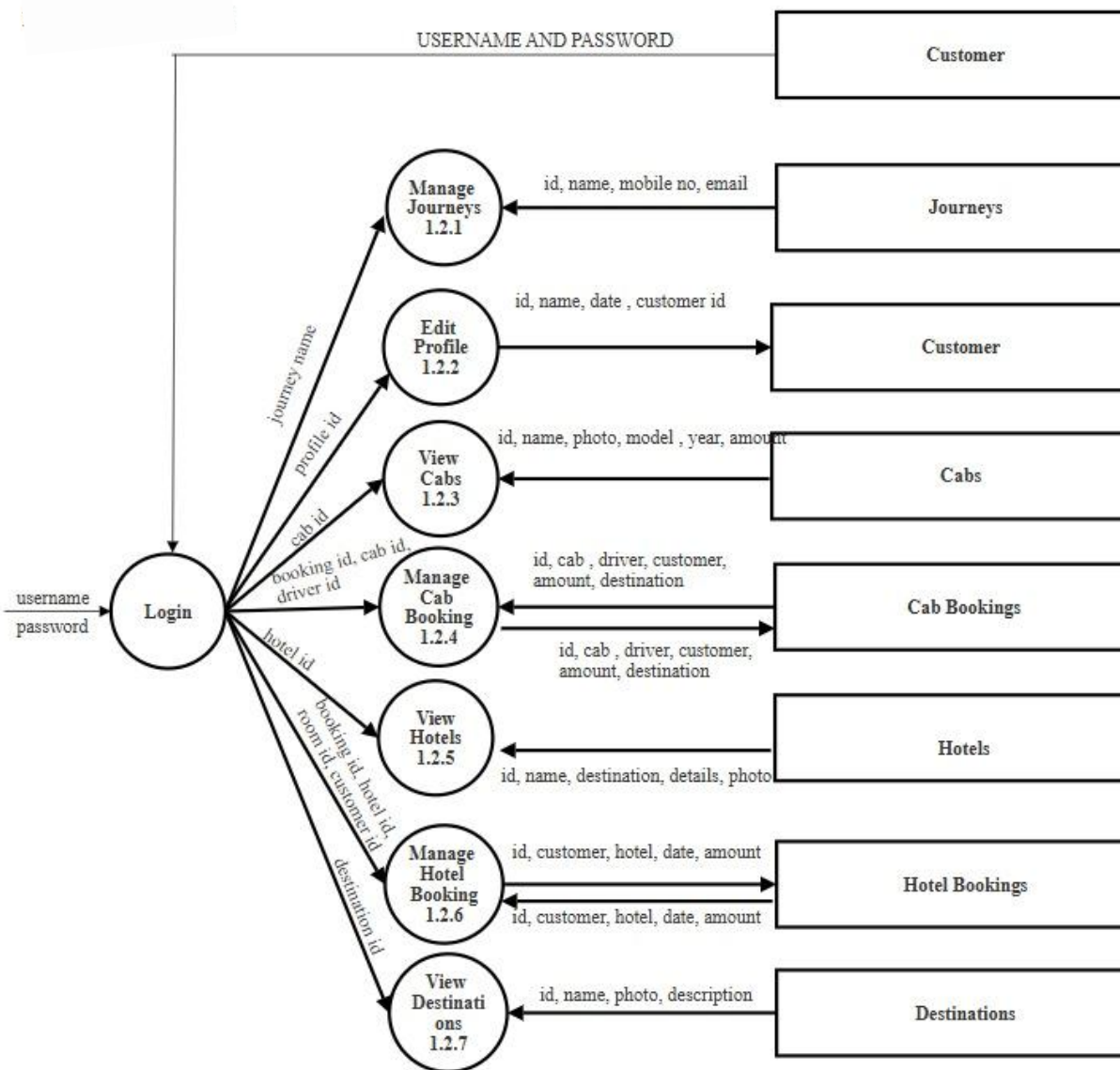
LEVEL-1



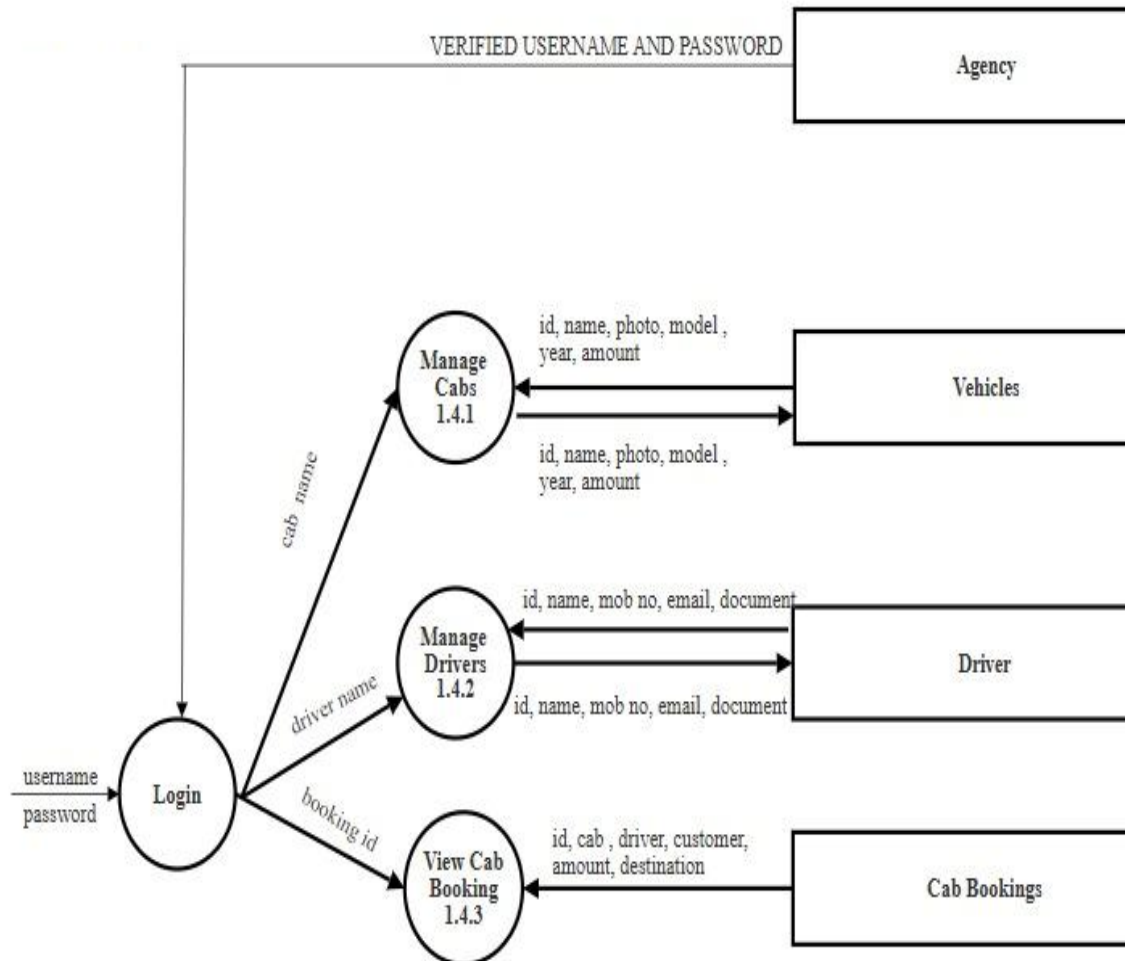
LEVEL-1.1 OF ADMIN

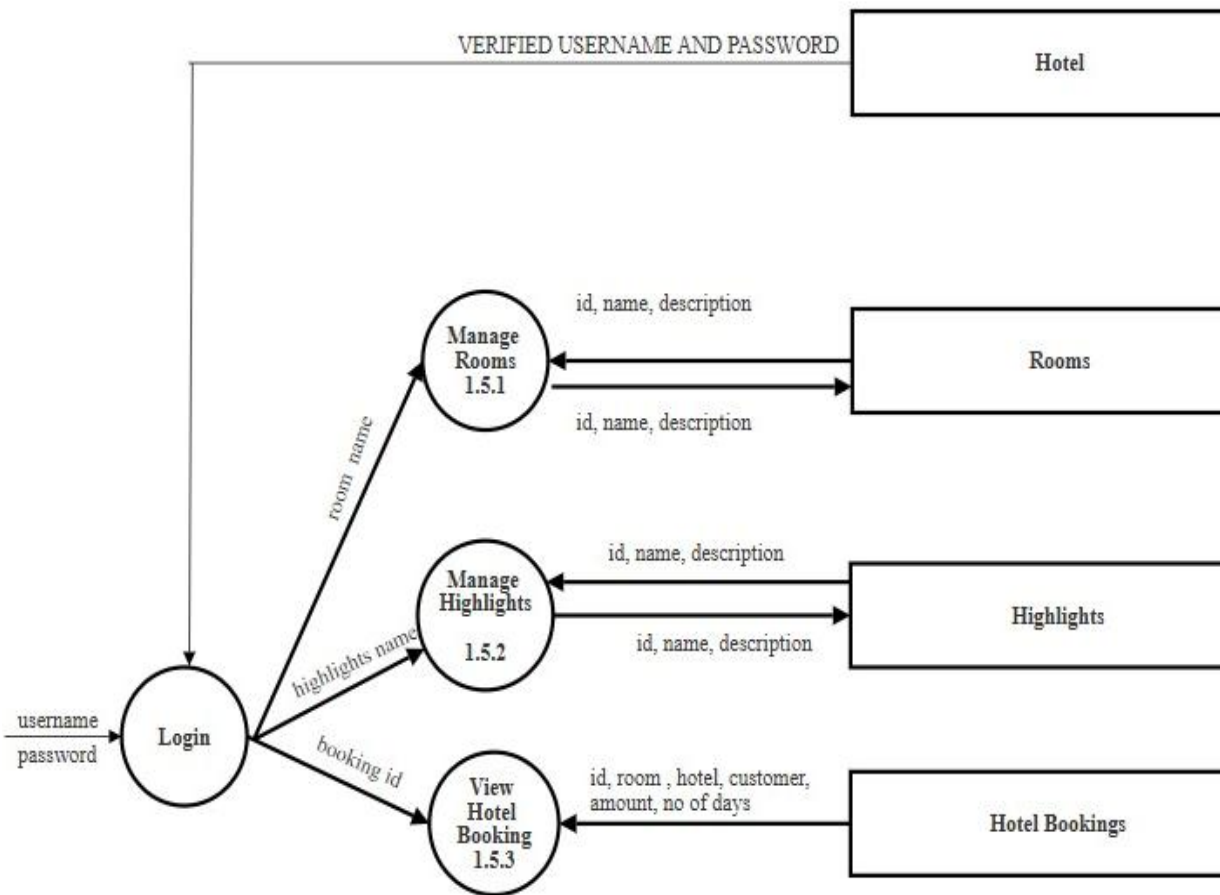


LEVEL-1.2 OF CUSTOMER



LEVEL-1.4 OF AGENCY



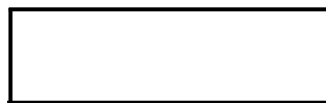
LEVEL-1.5 OF HOTEL

3.4.1 Entity relationship model

An entity relationship diagram is a specialized graphic that illustrates the inter-relationship between entities in a database. ER diagrams often use symbols to represent three different types of information. Boxes are commonly used to represent entities. Diamonds are normally used to represent relationships and ovals are used to represent attributes. An entity– relationship model is the result of using a systematic process to describe and define a subject area of business data. It does not define business process; only visualize business data. The data is represented as components (entities) that are linked with each other by relationships that express the dependencies and requirements between them, such as: one building may be divided into zero or more apartments, but one apartment can only be located in one building. Entities may have various properties (attributes) that characterize them. Diagrams created to represent these entities, attributes, and relationships graphically are called entity–relationship diagrams.

Symbols used in ER diagram: -

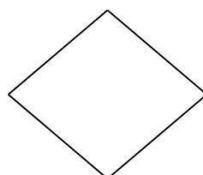
Entity:



Attributes:



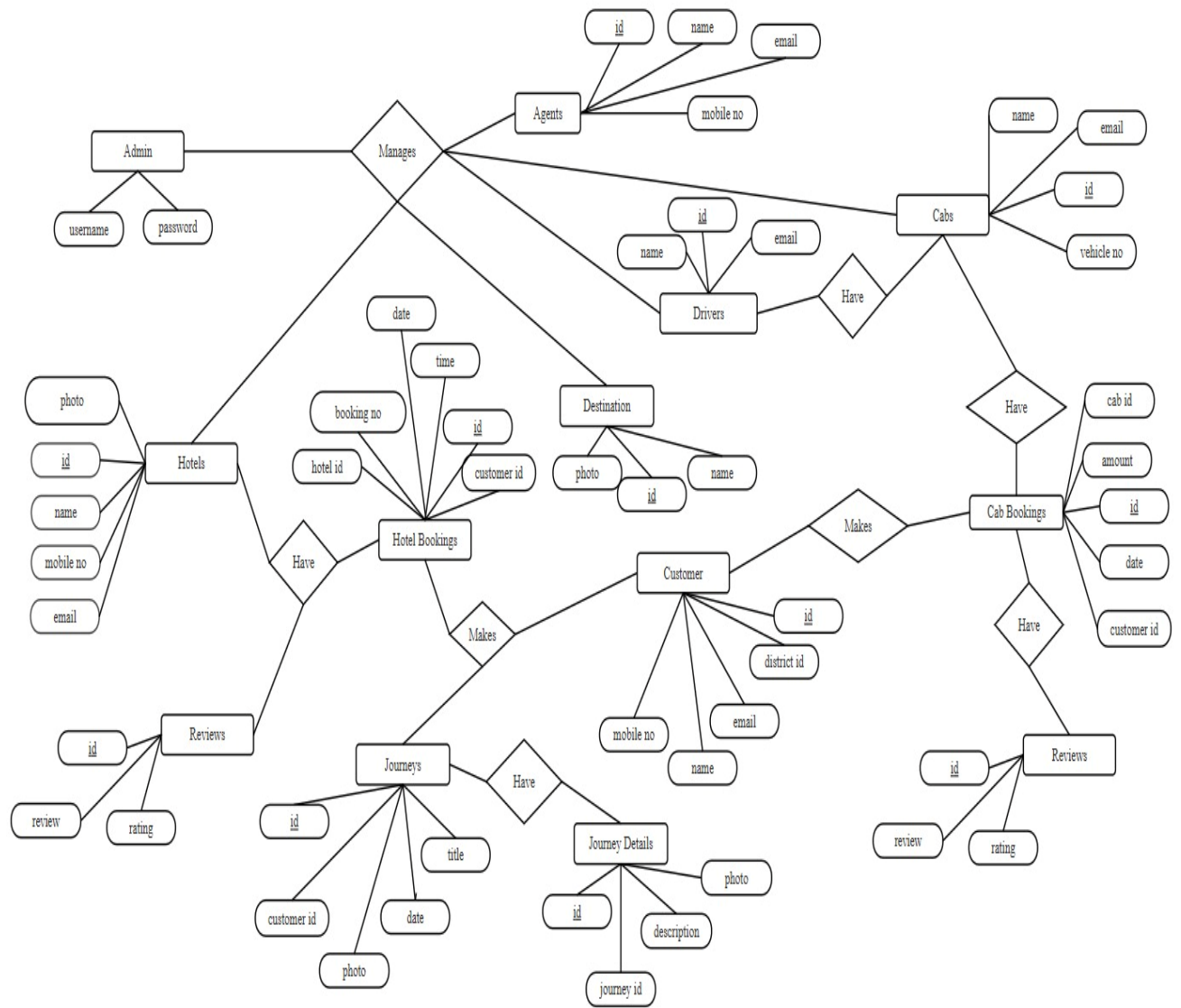
Relationship:



Arrow:



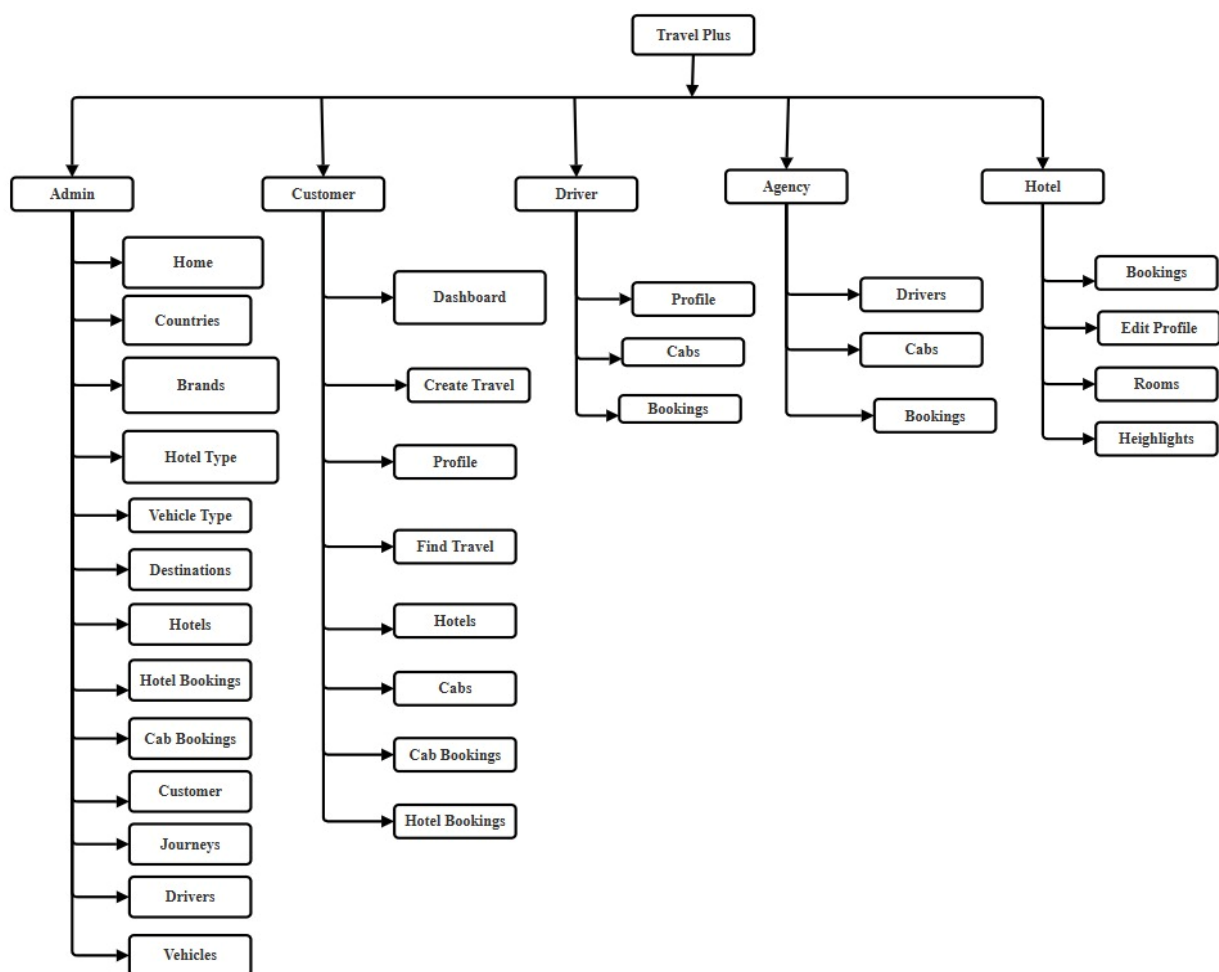
Entity Relationships Diagram



3.5 ARCHITECTURAL DESIGN

3.5.1 Structure Chart

A structure chart (SC) in software engineering and organizational theory, is a chart which shows the breakdown of a system to its lowest manageable levels. They are used in structure programs to arrange modules into a tree. Each module is represented by a box, which contains the module's name. The tree structure visualizes the relationships between the modules.



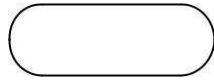
3.6 PROCEDURAL DESIGN

3.6.1 Flow Chart

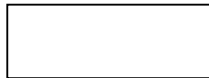
A flowchart is a type of diagram that represents an algorithm, workflow or process, showing the steps as boxes of various kinds, and their order by connecting them with arrows. This diagrammatic representation illustrates a solution model to a given problem. Flowcharts are used in analysing, designing, documenting or managing a process or program in various fields Symbols used:.

Flow line: 

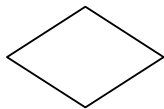
Terminal:



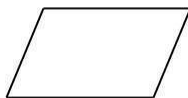
Process:



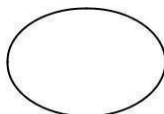
Decision:

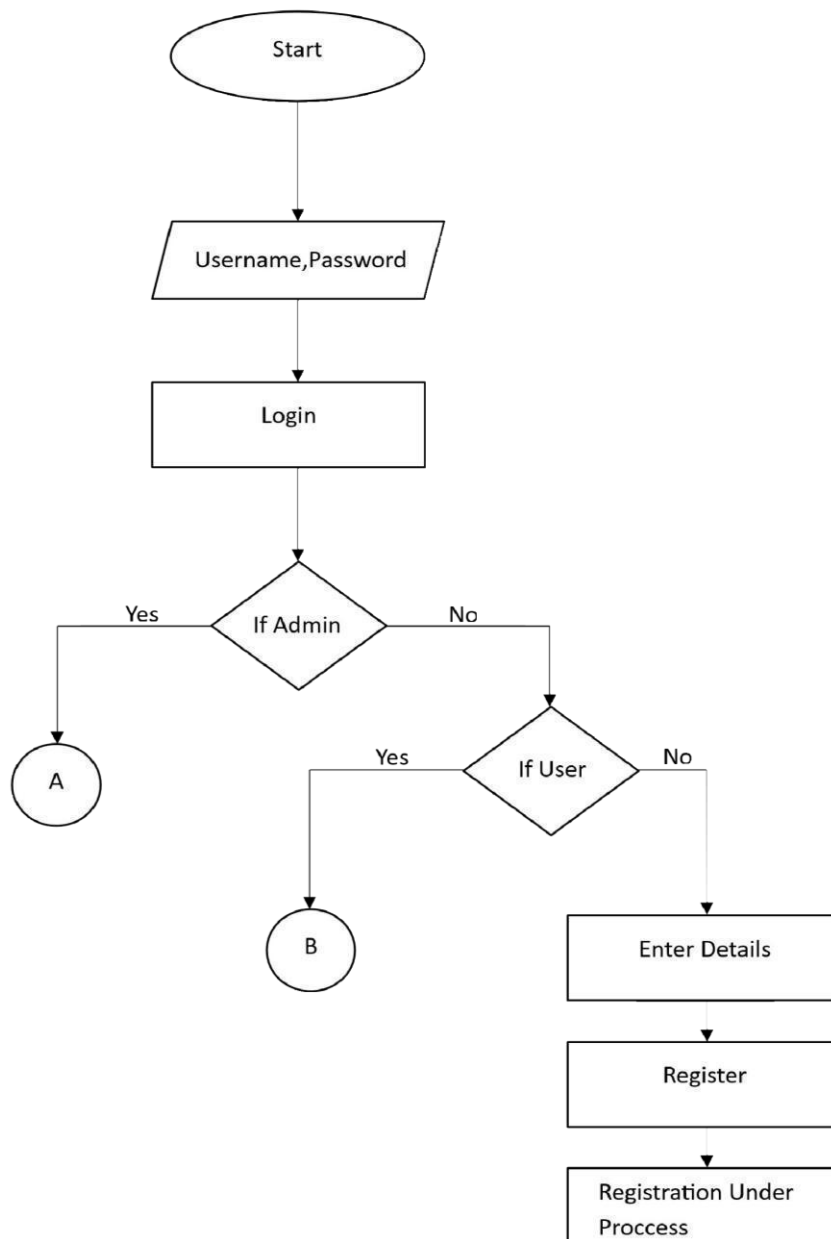


Input/Output:

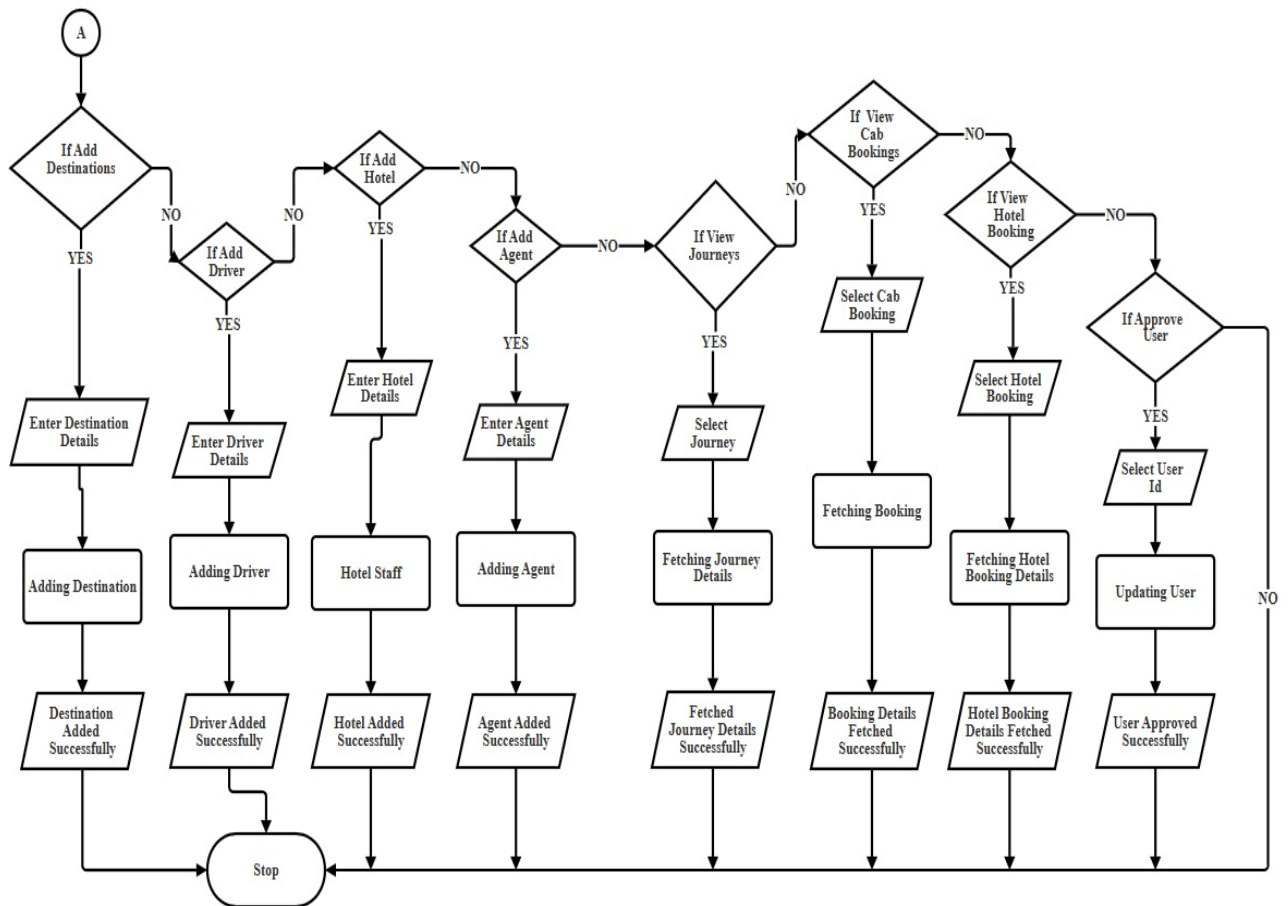


Connector:

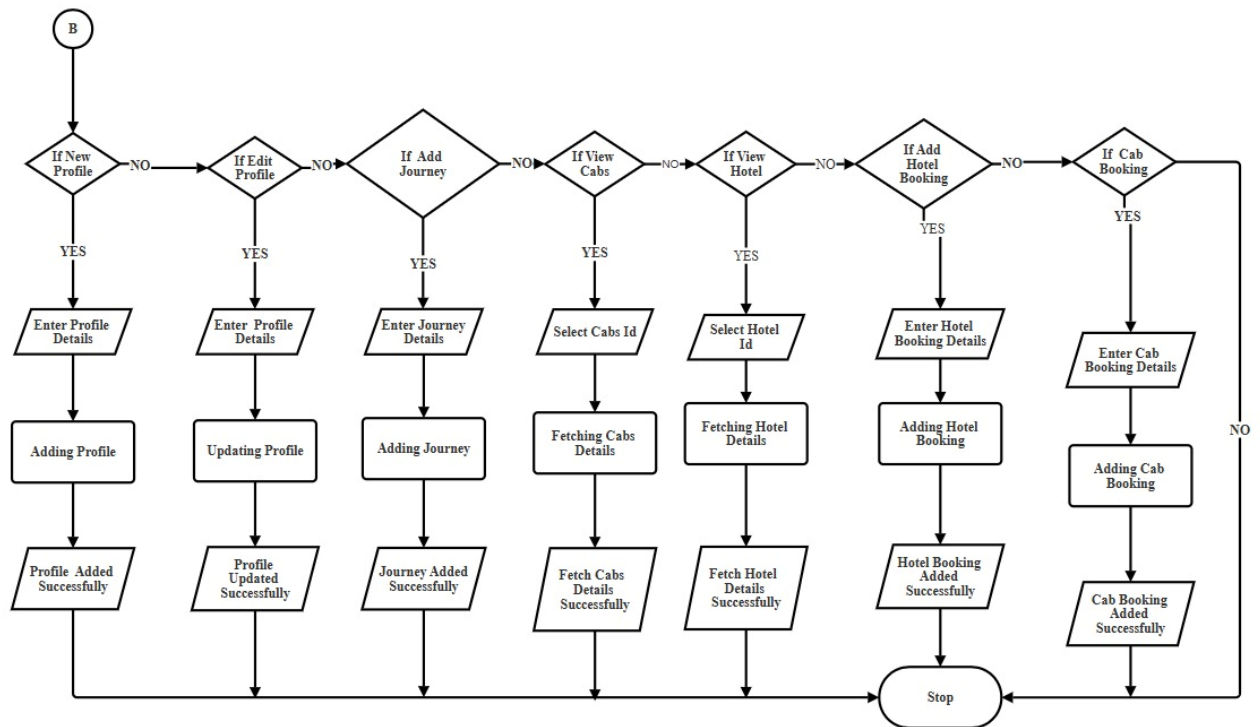


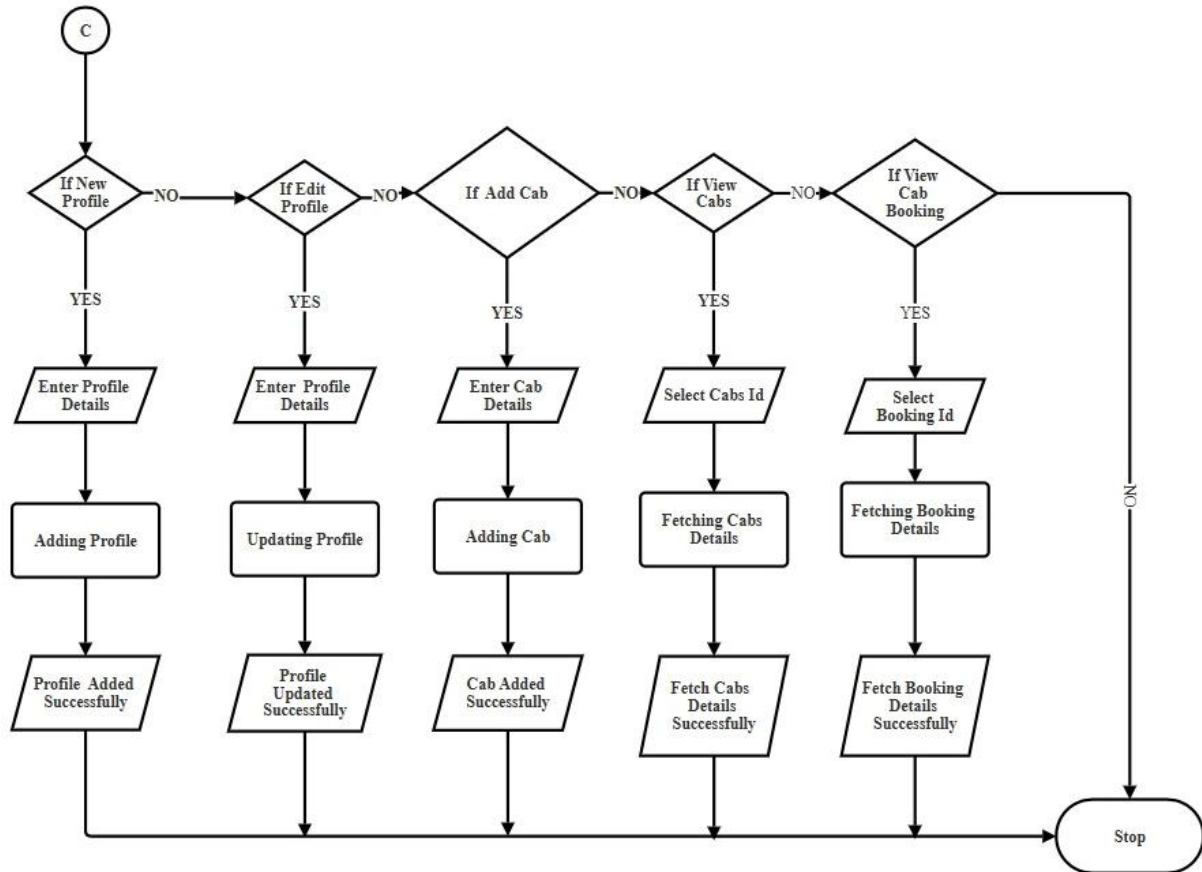
Flowchart

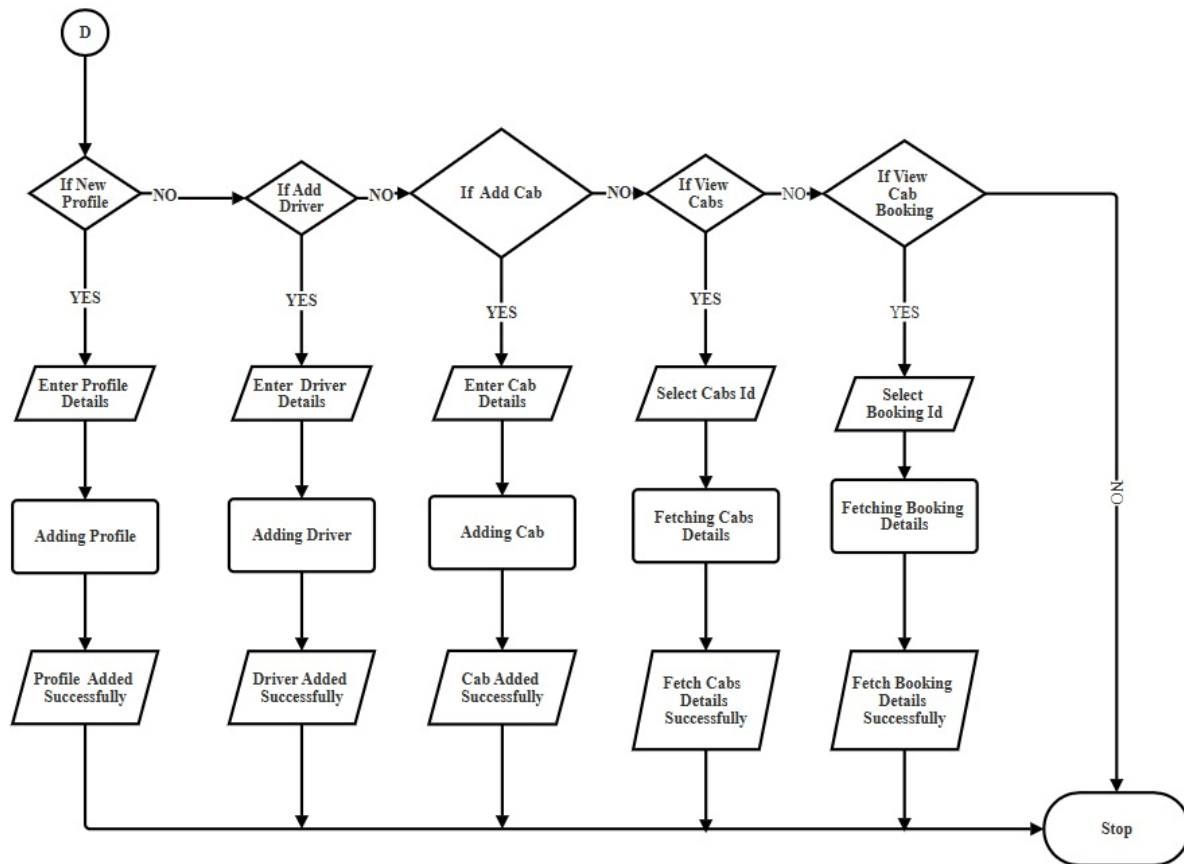
FLOWCHART OF ADMIN



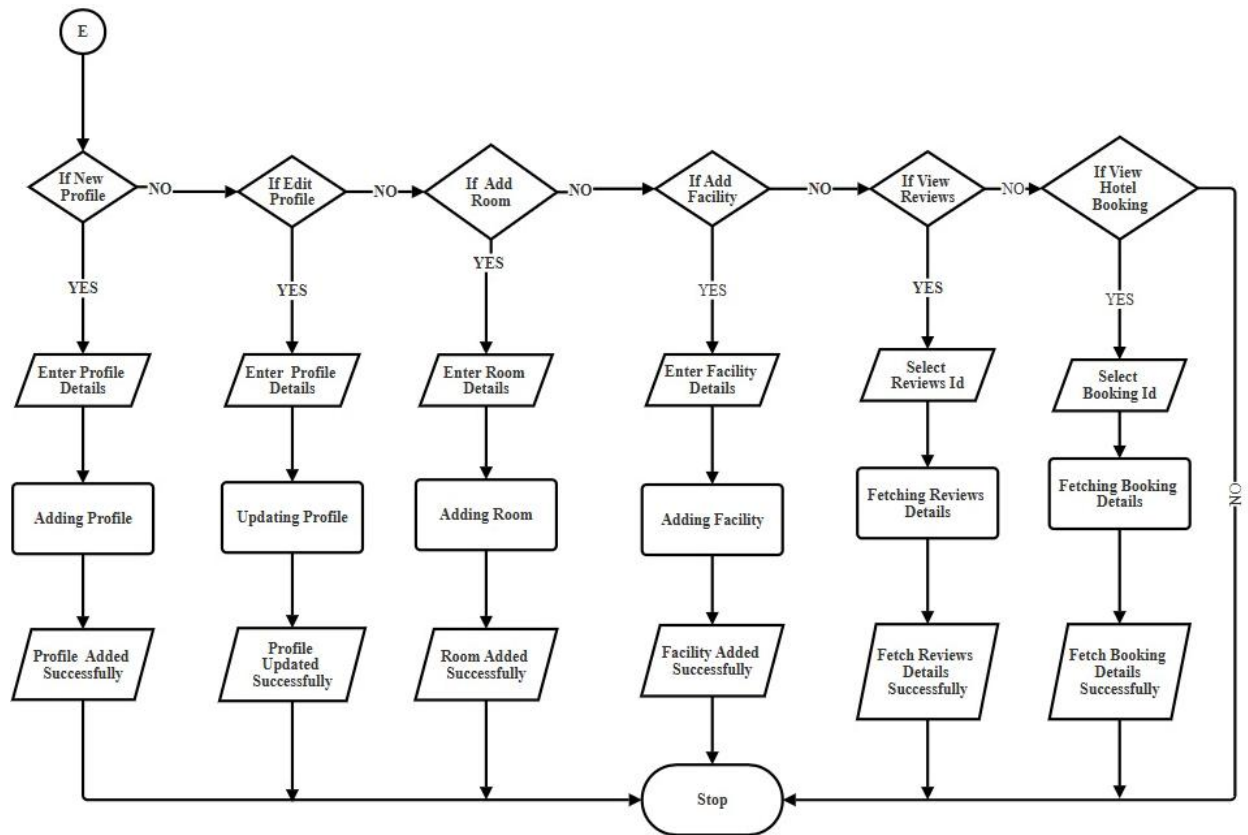
FLOWCHART OF CUSTOMER



FLOWCHART OF DRIVER

FLOWCHART OF AGENCY

FLOWCHART OF HOTEL



PROCESSING ENVIRONMENT

4.1 HARDWARE SPECIFICATION

Intel core i7 Processor

An Intel Core i7 is the fastest version of the Intel processor for consumer-end computers and devices. Like the Intel Core i5, the Core i7 is embedded with Intel Turbo Boost Technology. The Intel Core i7 is available in two- to six-core varieties, and can support up to 12 different threads simultaneously. Its processor clock speed ranges from 1.70 GHz to up to 3.90 GHz, with cache memory from 4 to 12 MB. Intel Core i7 thermal design power (TDP) range goes from 130 watts TDP to as low as 15 watts TDP. A Core i7 will typically be better for multitasking, media-editing and media-creation tasks, high-end gaming, and similar demanding workloads.

Processor speed

With technology, increased productivity goals, faster internet, and more devices, we've created a need for speed wherever we go. We're used to getting results instantaneously and expect our devices to keep up with our requests as we multi-task our way through life. Computer processors and their clock speed are two features we most commonly associate with high performing, fast technology. Computer processor speed (CPU speed) is one of the most important elements to consider when comparing computers.

RAM

RAM (Random Access Memory) is the hardware in a computing device where the operating system (OS), application programs and data in current use are kept so they can be quickly reached by the device's processor. RAM is the main memory in a computer. This memory stores a temporary data. Temporary data means as long as your computer system is running, your data is stored in RAM and as soon as your computer is shut down, the data stored in RAM gets deleted. RAM is volatile in nature, which means, the data is lost when the device is switched off.

Hard Disk Capacity

A computer hard disk drive (HDD) is a non-volatile data storage device. Non-volatile refers to storage devices that maintain stored data when turned off. All computers need a storage device, and Hard Disk Drive are just one example of a type of storage device. HDD are usually installed inside desktop computers, mobile devices, consumer electronics and enterprise storage arrays in data centres.

Keyboard

A multimedia keyboard is one with media keys — additional buttons, typically along the top, for controlling audio playback, for starting common applications (e.g., e-mail client and Web browser) and other auxiliary functionality. Many such keyboards also contain a volume knob, implemented as a rotary encoder so as to be able to provide relative volume changes regardless of the volume level set by the user in the operating system and application software.

Mouse

A mouse is a small device that a computer user pushes across a desk surface in order to point to a place on a display screen and to select one or more actions to take from that position.

USB

USB 2.0, also known as hi-speed USB was introduced in 2000. It is an updated version of USB 1.1, which provides improved functionalities and better speed. It is capable to deliver the maximum transfer speed of 480 Megabits per second. However, practically it is approximately 280 Mbps. USB 3.0, also known as SuperSpeed USB was first made available in November 2009. It is a much-improved version of USB 2.0

4.2 SOFTWARE SPECIFICATIONS

Windows 11

Windows 11 is the latest major release of Microsoft's Windows NT operating system, released in October 2021. It is a free upgrade to its predecessor, Windows 10 (2015), available for any Windows 10 devices that meet the new Windows 11 system requirements. Windows 11. A version of the Windows NT operating system.

React JS

React.js is a JavaScript library used for building user interfaces, particularly for single-page applications where dynamic data is frequently updated. Developed and maintained by Facebook, React allows developers to create reusable UI components that efficiently update and render when data changes.

Node JS

Node.js is an open-source, cross-platform, back-end JavaScript runtime environment that runs on a JavaScript Engine and executes JavaScript code outside a web browser, which was designed to build scalable network applications. Node.js is primarily used for nonblocking, event-driven servers, due to its single-threaded nature. It's used for traditional web sites and back-end API services, but was designed with real-time, push based architectures in mind.

MySQL

MySQL is a widely-used open-source relational database management system (RDBMS) that operates under the SQL (Structured Query Language) framework. It provides a robust platform for storing and managing structured data, making it ideal for applications requiring structured data storage, retrieval, and management. MySQL is known for its reliability, scalability, and ease of use, making it a popular choice for both small-scale applications and large-scale enterprise solutions.

Visual Studio Code

Visual Studio Code, also commonly referred to as VS Code, is a source-code editor made by Microsoft for Windows, Linux and macOS. Features include support for debugging, syntax highlighting, intelligent code completion, snippets, code refactoring, and embedded Git..

CODING AND IMPLEMENTATION

5.1 CODING

Authentication Business Logic (Authentication.bl)

```

using System;
using System.Collections.Generic;
using System.Data.Entity;
using System.Linq;
using System.Net;
using System.Runtime.Serialization;
using System.Security.Cryptography;
using System.Text;
using System.Threading.Tasks;

namespace TravelPro.Model
{
    public class AuthenticationBL
    {
        public static UserSession IsTokenValid(string token)
        {
            using (TravelProDB context = new TravelProDB())
            {
                var session = context.UserSessions
                    .FirstOrDefault(x => x.Token == token);

                if (session != null)
                {
                    // Optional expiry check
                    //if(session.SessionTimeStamp + TimeSpan.FromMinutes(session.ExpiresInMinutes)
>= DateTime.Now)
                    // {
                    return session;
                    // }
                }
                return null;
            }
        }

        public static UserSession LoginWithUserName(string userName, string password, out
bool? isVerified)
        {
            isVerified = null;
            UserSession userSession = null;

            using (TravelProDB context = new TravelProDB())
            {

```

```

var user = context.Users
    .FirstOrDefault(x => x.UserName == userName);

if (user != null)
{
    // Customer, Driver, Hotel, Agency must be verified
    if ((user.Role == "Customer" || user.Role == "Driver" ||
        user.Role == "Hotel" || user.Role == "Agency") && !user.IsVerified)
    {
        isVerified = false;
        return null;
    }

    isVerified = true;

    var storedPassword = Convert.FromBase64String(user.Password);

    if (ComparePasswords(storedPassword, Encoding.ASCII.GetBytes(password)))
    {
        string token = Convert.ToBase64String(Guid.NewGuid().ToArray());

        userSession = new UserSession
        {
            ExpiresInMinutes = 480,
            SessionTimeStamp = DateTime.Now,
            Token = token,
            UserId = user.Id
        };

        context.UserSessions.Add(userSession);
        context.SaveChanges();

        userSession.User = context.Users
            .FirstOrDefault(x => x.Id == userSession.UserId);

        return userSession;
    }
    else
    {
        throw new WebFaultException(HttpStatusCode.Forbidden);
    }
}

return null;
}

```



```

public static bool LogOutWithToken(string token)
{
    using (TravelProDB context = new TravelProDB())
    {
        var userSession = context.UserSessions
            .FirstOrDefault(x => x.Token == token);

        if (userSession != null)
        {
            userSession.UserSessionStatus = UserSessionStatus.LoggedOut;
            userSession.SessionTimeStamp = DateTime.Now;

            context.Entry(userSession).State = EntityState.Modified;
            context.SaveChanges();
        }

        return true;
    }
}

// Create salt
public static byte[] CreatePasswordSalt(byte[] unsaltedPassword)
{
    byte[] saltValue = new byte[64];

    using (RNGCryptoServiceProvider rng = new RNGCryptoServiceProvider())
    {
        rng.GetBytes(saltValue);
    }

    return saltValue;
}

// Create salted password
public static byte[] CreateSaltedPassword(byte[] saltValue, byte[] unsaltedPassword)
{
    byte[] rawSalted = new byte[unsaltedPassword.Length + saltValue.Length];

    unsaltedPassword.CopyTo(rawSalted, 0);
    saltValue.CopyTo(rawSalted, unsaltedPassword.Length);

    using (SHA1 sha1 = SHA1.Create())
    {
        byte[] saltedPassword = sha1.ComputeHash(rawSalted);
    }
}

```

```

        byte[] dbPassword = new byte[saltedPassword.Length + saltValue.Length];
        saltedPassword.CopyTo(dbPassword, 0);
        saltValue.CopyTo(dbPassword, saltedPassword.Length);

        return dbPassword;
    }
}

// Compare passwords
private static bool ComparePasswords(byte[] storedPassword, byte[] hashedPassword)
{
    if (storedPassword == null || hashedPassword == null)
        return false;

    byte[] saltValue = new byte[64];
    int saltOffset = storedPassword.Length - 64;

    for (int i = 0; i < 64; i++)
    {
        saltValue[i] = storedPassword[saltOffset + i];
    }

    byte[] saltedPassword = CreateSaltedPassword(saltValue, hashedPassword);

    return CompareByteArray(storedPassword, saltedPassword);
}

// Compare byte arrays
private static bool CompareByteArray(byte[] array1, byte[] array2)
{
    if (array1.Length != array2.Length)
        return false;

    for (int i = 0; i < array1.Length; i++)
    {
        if (array1[i] != array2[i])
            return false;
    }

    return true;
}

[Serializable]
internal class WebFaultException : Exception
{
    private HttpStatusCode forbidden;

```

```

public WebFaultException() { }

public WebFaultException(string message) : base(message) { }

public WebFaultException(HttpStatusCode forbidden)
{
    this.forbidden = forbidden;
}

public WebFaultException(string message, Exception innerException)
    : base(message, innerException) { }

protected WebFaultException(SerializationInfo info, StreamingContext context)
    : base(info, context) { }
}
}
}

```

Cab Booking Model (CabBooking.cs)

```

using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace TravelPro.Model
{
    public class CabBooking
    {
        public long Id { get; set; }

        public DateTime Date { get; set; }
        public bool IsActive { get; set; }

        [StringLength(100)]
        public string Time { get; set; }

        public float TotalKmS { get; set; }
        public float TotalAmount { get; set; }
        public int Rating { get; set; }

        [StringLength(500)]

```

```

public string Review { get; set; }

[StringLength(100)]
public string Status { get; set; }

[StringLength(100)]
public string LocationFrom { get; set; }

[StringLength(100)]
public string LocationTo { get; set; }

public long CustomerId { get; set; }
public long VehicleId { get; set; }
public long DriverId { get; set; }
public long DestinationId { get; set; }

public virtual Customer Customer { get; set; }
public virtual Vehicle Vehicle { get; set; }
public virtual Driver Driver { get; set; }
public virtual Destination Destination { get; set; }
}
}

```

User Model (user.cs)

```

using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace TravelPro.Model
{
    public class User
    {
        public long Id { get; set; }

        [StringLength(150)]
        public string UserName { get; set; }

        [StringLength(200)]
        public string Password { get; set; }

        [StringLength(300)]

```

```

public string PasswordSalt { get; set; }

[StringLength(100)]
public string Role { get; set; }

public bool IsBlocked { get; set; }
public bool IsVerified { get; set; }

public long? CustomerId { get; set; }
public virtual Customer Customer { get; set; }

public long? DriverId { get; set; }
public virtual Driver Driver { get; set; }

public long? HotelId { get; set; }
public virtual Hotel Hotel { get; set; }

public long? AgencyId { get; set; }
public virtual Agency Agency { get; set; }
}
}

```

Hotel Booking Model (HotelBooking.cs)

```

using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace TravelPro.Model
{
    public class HotelBooking
    {
        public long Id { get; set; }

        public int Adults { get; set; }
        public int Kids { get; set; }
        public int TotalDays { get; set; }
        public float Total { get; set; }

        public DateTime Date { get; set; }
        public DateTime FromDate { get; set; }
        public DateTime ToDate { get; set; }
    }
}

```

```

public long HotelId { get; set; }
public virtual Hotel Hotel { get; set; }

public long HotelRoomId { get; set; }
public virtual HotelRoom HotelRoom { get; set; }

public long CustomerId { get; set; }
public virtual Customer Customer { get; set; }

[StringLength(500)]
public string Review { get; set; }

public int Rating { get; set; }
public bool IsActive { get; set; }
}
}

```

Journey Model (Journey.cs)

```

using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace TravelPro.Model
{
    public class HotelBooking
    {
        public long Id { get; set; }

        public int Adults { get; set; }
        public int Kids { get; set; }
        public int TotalDays { get; set; }
        public float Total { get; set; }

        public DateTime Date { get; set; }
        public DateTime FromDate { get; set; }
        public DateTime ToDate { get; set; }

        public long HotelId { get; set; }
        public virtual Hotel Hotel { get; set; }
        public long HotelRoomId { get; set; }
    }
}

```

```

public virtual HotelRoom HotelRoom { get; set; }

public long CustomerId { get; set; }
public virtual Customer Customer { get; set; }

[StringLength(500)]
public string Review { get; set; }

public int Rating { get; set; }
public bool IsActive { get; set; }
}
}

```

Database Context (TravelProDB.cs)

```

using System;
using System.Data.Entity;
using System.Linq;

namespace TravelPro.Model
{
    public class TravelProDB : DbContext
    {
        public TravelProDB()
            : base("name=TravelProDB")
        {
        }

        public virtual DbSet<Country> Countries { get; set; }
        public virtual DbSet<Destination> Destinations { get; set; }
        public virtual DbSet<Customer> Customers { get; set; }

        public virtual DbSet<Journey> Journeys { get; set; }
        public virtual DbSet<JourneyDestination> JourneyDestinations { get; set; }
        public virtual DbSet<JourneyDetail> JourneyDetails { get; set; }
        public virtual DbSet<JourneyCab> JourneyCabs { get; set; }
        public virtual DbSet<JourneyHotel> JourneyHotels { get; set; }
        public virtual DbSet<JourneyDirectory> JourneyDirectories { get; set; }
        public virtual DbSet<JourneyGallery> JourneyGalleries { get; set; }

        public virtual DbSet<VehicleType> VehicleTypes { get; set; }
        public virtual DbSet<Vehicle> Vehicles { get; set; }
        public virtual DbSet<Brand> Brands { get; set; }
        public virtual DbSet<Driver> Drivers { get; set; }
        public virtual DbSet<CabBooking> CabBookings { get; set; }
    }
}

```

```

    public virtual DbSet<HotelType> HotelTypes { get; set; }
    public virtual DbSet<Hotel> Hotels { get; set; }
    public virtual DbSet<HotelHighlight> HotelHighlights { get; set; }
    public virtual DbSet<HotelRoom> HotelRooms { get; set; }
    public virtual DbSet<HotelBooking> HotelBookings { get; set; }

    public virtual DbSet<User> Users { get; set; }
    public virtual DbSet<UserSession> UserSessions { get; set; }

    public virtual DbSet<Agency> Agencies { get; set; }
}
}

```

Driver Model(Driver.cs)

```

using System;
using System.Data.Entity;
using System.Linq;

namespace TravelPro.Model
{
    public class TravelProDB : DbContext
    {
        public TravelProDB()
            : base("name=TravelProDB")
        {
        }

        public virtual DbSet<Country> Countries { get; set; }
        public virtual DbSet<Destination> Destinations { get; set; }
        public virtual DbSet<Customer> Customers { get; set; }

        public virtual DbSet<Journey> Journeys { get; set; }
        public virtual DbSet<JourneyDestination> JourneyDestinations { get; set; }
        public virtual DbSet<JourneyDetail> JourneyDetails { get; set; }
        public virtual DbSet<JourneyCab> JourneyCabs { get; set; }
        public virtual DbSet<JourneyHotel> JourneyHotels { get; set; }
        public virtual DbSet<JourneyDirectory> JourneyDirectories { get; set; }
        public virtual DbSet<JourneyGallery> JourneyGalleries { get; set; }

        public virtual DbSet<VehicleType> VehicleTypes { get; set; }
        public virtual DbSet<Vehicle> Vehicles { get; set; }
        public virtual DbSet<Brand> Brands { get; set; }
        public virtual DbSet<Driver> Drivers { get; set; }
        public virtual DbSet<CabBooking> CabBookings { get; set; }
    }
}

```



```

    public virtual DbSet<HotelType> HotelTypes { get; set; }
    public virtual DbSet<Hotel> Hotels { get; set; }
    public virtual DbSet<HotelHighlight> HotelHighlights { get; set; }
    public virtual DbSet<HotelRoom> HotelRooms { get; set; }
    public virtual DbSet<HotelBooking> HotelBookings { get; set; }

    public virtual DbSet<User> Users { get; set; }
    public virtual DbSet<UserSession> UserSessions { get; set; }

    public virtual DbSet<Agency> Agencies { get; set; }
}
}

```

Hotel Model(hotel.cs)

```

using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace TravelPro.Model
{
    public class Hotel
    {
        public long Id { get; set; }
        [StringLength(100)]
        public string Name { get; set; }
        [StringLength(100)]
        public string Email { get; set; }
        [StringLength(100)]
        public string MobileNo { get; set; }
        public long DestinationId { get; set; }
        public decimal CostPerDay { get; set; }

        [StringLength(200)]
        public string Image1 { get; set; }
        [StringLength(200)]
        public string Image2 { get; set; }
        [StringLength(200)]
        public string Image3 { get; set; }
        [StringLength(200)]
        public string Image4 { get; set; }
        [StringLength(200)]
    }
}

```

```
public string Image5 { get; set; }

[StringLength(200)]
public string Location { get; set; }
[StringLength(200)]
public string Address { get; set; }
public long HotelTypeId { get; set; }

public int StarRating { get; set; }
public decimal AvgRating { get; set; }

public virtual Destination Destination { get; set; }
public virtual HotelType HotelType { get; set; }
public bool IsActive { get; set; }
[StringLength(200)]
public string RegistrationCertificate { get; set; }
}
}
```

5.2 TESTING

Testing is the process of detecting errors. Testing performs a very critical role for quality assurance and for ensuring the reliability of software. The results of testing are used later on during maintenance also. The aim of testing is often to demonstrate that a program works by showing that it has no errors. The basic purpose of testing phase is to detect the errors that may be present in the program. Hence one should not start testing with the intent of showing that a program works, but the intent should be to show that a program doesn't work. Testing is the process of executing a program with the intent of finding errors.

5.2.1 Unit Testing

Unit testing focuses verification effort on the smallest unit of software i.e., the module. Using the detailed design and the process specifications, unit testing is done to uncover errors within the boundary of the module. All modules must be successful in the unit test before the integration testing starts. In this project, each service can be thought of a module. There are 2 modules like Admin, User. While developing the modules, unit testing is done to ensure that each module works without any error. The inputs are validated after accepting from the user. In this project, developer tests the programs up as system. Software units in a system are the modules and routines that are assembled and integrated to form a specific function. Unit testing is first done on modules, independent of one another to locate errors. This enables to detect errors. Through these, errors resulting from interaction between modules are initially avoided.

Black Box Testing: In the black box testing, test cases are designed from the examination of the input/output values only and no knowledge of design or code is required. The main approaches are:

- Equivalent class partitioning
- Boundary value analysis

White Box Testing: White box testing is an important type of unit testing. A large number of white box testing strategies exist. White box testing strategies are:

- Fault based testing

5.2.2 Integration Testing

Integration testing is a type of software testing in which the different units, modules or components of a software application are tested as a combined entity. However, these modules may be coded by different programmers. The aim of integration testing is to test the interfaces between the modules and expose any defects that may arise when these components are integrated and need to interact with each other.

Types of test cases:

Integration test case: Here we execute test cases which just tell about the connectivity from one module to other module and integrating one application to other application and how the application moves from parent node to child no demand vice versa.

Functional Test case: Here we execute test cases which tell about the functionality of the application and talks about the desired output to be seen. Internally we have different type of test cases when we write here like range, output values, BVA, ECP and so on. We give input and expect some output according to the SRS. Here we check individual module completely by checking each and every tab, text box, and buttons and so on.

Non-Functional Test case: Test cases related to user friendliness like font, image, color, how easy to navigate etc., performance related, security related comes under here.

User Acceptance test case: These test cases are crucial and very important to client-side people, because these test case talks about these business and approach of the application to complete a particular client task, which is also called as End to End Business scenario test case. Here we won't be doing testing related to UI, Functional or Non-Functional, we talk about business and scenario for which the application is made for.

5.2.3 System Testing

Here the entire software system is tested. The reference document for this process is the requirements document, and the goal is to see if software meets its requirements. The entire software has been tested against requirements of project and it is checked whether all requirements of project have been satisfied or not.

Alpha Test: The first test of newly developed, when the first round of bugs has been fixed, the product goes with actual users for testing. For custom software the customer may be invited into the vendor's facilities for an alpha test to ensure the client's vision has been interpreted properly by the developer.

Beta Test: A test of new or revised software application that is performed by users at their facilities under normal operating conditions. Beta testing follows alpha testing. Vendors of packaged software often offer their customers the opportunity of beta testing new releases or versions and the beta testing of elaborate products such as operating systems can take months.

Acceptance testing: Testing is performed by the Client of the application to determine whether the application is developed as per the requirements specified by him/her. It is performed within the development of the organization or at the client site. In our project cab rental system, the developer uses unit testing to ensure the overall working of the system.

SECURITY, BACKUP AND RECOVERY MECHANISM

Security

This application deals with a simple authorization mechanism i.e., username and password. The user just must register into the application with his username and password. The application can only be operated by a registered user and with his account only. Admin can accept or decline the user request based on the authenticity of the data entered. Security in "Merch Mingle" includes user authentication, data encryption, firewalls, intrusion detection, and regular audits to protect user information. Compliance with PCI DSS ensures secure payment processing. These measures safeguard data and maintain the website's integrity against potential threats and breaches.

Backup

In any case where the user encounters a problem with the application, support will be provided to the user for fixing the problem. Backup source code for our application is always kept.

"Merch Mingle" employs data backups, redundancy strategies, and a disaster recovery plan. Automated backups preserve data integrity. Redundancy ensures high availability, while the disaster recovery plan outlines steps for system restoration in unforeseen events, ensuring service continuity.

Recovery

In any case the application crashes the application just needs to be refreshed to be back online. In the worst case refreshing the system might prove helpful and solves the problem. If the application has any problems, there are other recovery options as well. "Merch Mingle" utilizes automated backups, storing critical data offsite in secure locations to mitigate the risk of data loss. Version control tracks code changes, enabling easy rollbacks. A data retention policy ensures efficient storage management, enhancing data security and system resilience.

FUTURE ENHANCEMENT

In the future, Travel Pro aims to evolve into a fully intelligent and user-centric travel management platform by introducing several innovative enhancements that elevate both convenience and service quality. The system will expand its offerings to include a wider variety of hotels and vehicles, providing more flexibility for users based on preferences such as comfort level, amenities, seating capacity, and price range. To make searching and booking more efficient, advanced filtering features will be implemented, allowing users to sort hotels and vehicles by location, type, price, ratings, fuel type, air conditioning, and purpose of travel, such as leisure trips, family vacations, or business travel. One-way vehicle bookings will also be introduced, enabling users to pick up a car from one location and drop it off at another, providing greater freedom for inter-city or airport travel. Guided travel packages will be offered, including pre-planned itineraries and scenic routes, allowing users to enjoy curated experiences with recommendations for local attractions, accommodations, and transportation. In terms of safety and monitoring, real-time GPS tracking will be integrated for vehicles, while travel updates and alerts will be provided for hotels and destinations, enhancing user trust and travel security. AI-based recommendation systems will be embedded into the platform to suggest the best itineraries, hotels, and vehicles based on the user's travel history, preferences, location, and even weather conditions. The overall system interface will continue to evolve for multi-device compatibility, ensuring a seamless and intuitive booking experience across smartphones, tablets, and desktops.

With these forward-looking enhancements, Travel Pro is set to become not just a travel management platform, but a complete travel companion that adapts to modern user needs with intelligence, flexibility, and reliability.

SOFTWARE MAINTENANCE

Software maintenance is widely accepted part of SDLC now days. It stands for all the modifications and update done after the delivery of software product. Software Maintenance planning includes ten activities:

- **Modification** – After the application has become the responsibility of the maintenance team, explain how to analyse each request; confirm and check validity; investigate and propose solutions; document the proposal and get the required authorizations to apply the modifications.
- **Implementation** – Describe the process for considering the implementation of the modification itself.
- **Acceptance** – Describe how the modification is accepted by the maintenance team
- **Migration** – Describe any migration tasks that need to be executed. If the software needs to be moved to another system, outline the steps to do so without impacting its functionality.
- **Transition** – Document the sequence of activities to transition the system from Development to Maintenance.
- **Service Level Agreements** – Document SLAs and maintenance contracts negotiated by Maintenance.
- **Change Request** – Outline the problem-handling process to prioritize, documents and route change and maintenance requests.
- **Modification Request acceptance/reject**– Describe the request including details of the size/effort/complexity. If this is too complex to resolve, outline the steps to route the issue back to the software team.
- **Retirement** – This is the final stage in the lifecycle. Describe how to retire the software and the steps to archive any data that may be a by-product of the system.

Types of Maintenance

Traditionally, 5 types of maintenance have been distinguished, which are differentiated by the nature of the tasks that they include:

Corrective maintenance: The set of tasks is destined to correct the defects to be found in the different equipment and that are communicated to the maintenance department by users of the same equipment.

Preventive Maintenance: Its mission is to maintain a level of certain service on equipment, programming the interventions of their vulnerabilities in the most opportune time. It is used to be a systematic character, that is, the equipment is inspected even if it has not given any symptoms of having a problem.

Predictive Maintenance: It constantly pursues know and report the status and operational capacity of the installations by knowing the values of certain variables, which represent such state and operational ability. To apply this maintenance, it is necessary to identify physical variables (temperature, vibration, power consumption, etc.) and to understand which type of variation is indicative of problems that may be appearing on the equipment. This maintenance is the most technical, since it requires advanced technical resources, and at times of strong mathematical, physical and / or technical knowledge.

Zero Hours Maintenance (Overhaul): The set of tasks whose goal is to review the equipment at scheduled intervals before appearing any failure, when the reliability of the equipment has decreased considerably so it is risky to make forecasts of production capacity. This review is based on leaving the equipment to zero hours of operation, that is, as if the equipment were new. These reviews will replace or repair all items subject to wear.

CONCLUSION

The Travel Pro project introduces a comprehensive solution aimed at modernizing and optimizing the travel planning and management process. Through meticulous design and implementation, Travel Pro offers a robust system comprising three crucial modules: Admin, User, and Service Providers (Hotels and Cars).

The Admin module equips platform administrators with a suite of versatile functionalities, facilitating efficient management of destinations, hotels, vehicles, user registrations, booking requests, and feedback. This interface provides a comprehensive toolkit to streamline workflows, maintain data integrity, and enhance operational efficiency.

The User module offers travelers an intuitive and seamless interface to interact with the system. With features such as secure registration, login, exploration of destinations, hotel and car search, booking, itinerary management, feedback submission, and booking history tracking, users can effortlessly navigate the platform from planning to confirmation, ensuring a smooth and convenient travel experience.

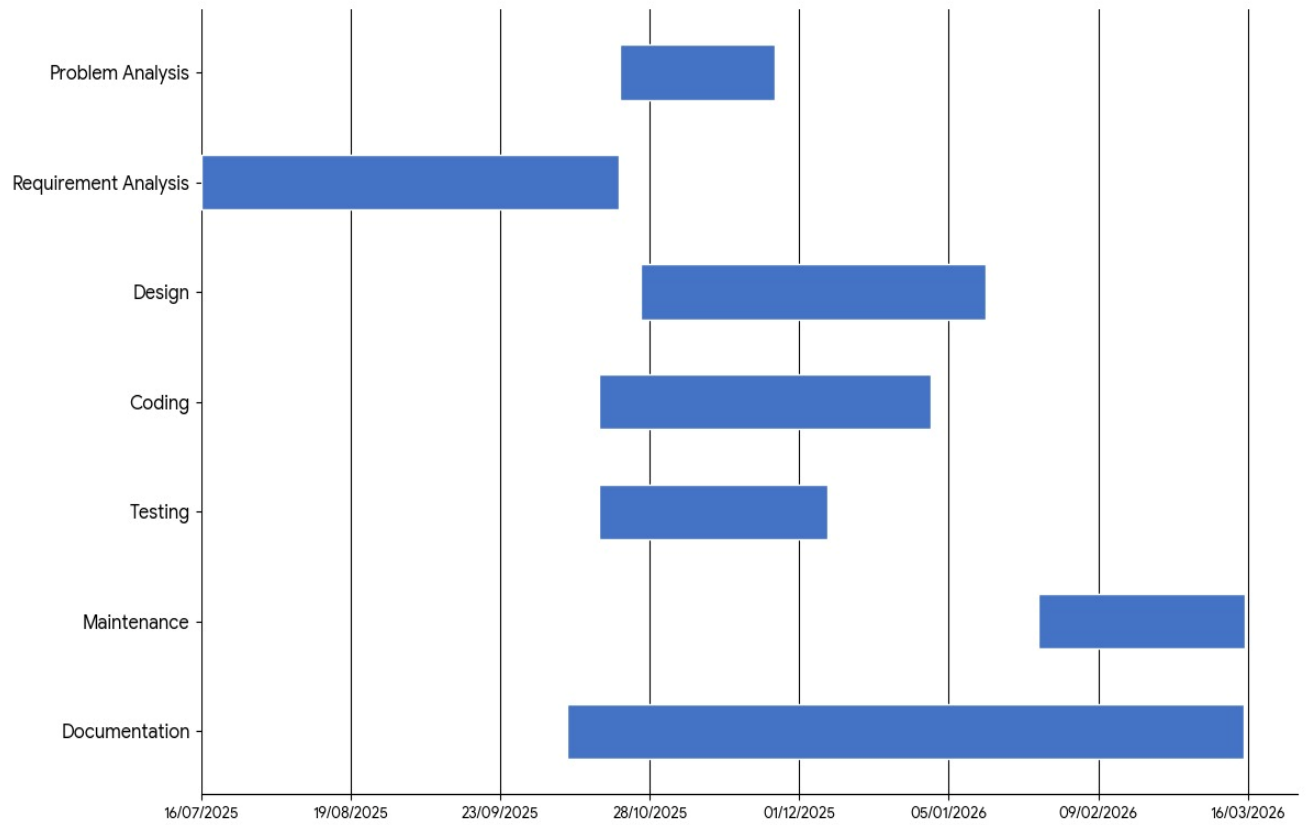
The Service Provider modules allow hotel and car providers to manage their services effectively. Providers can add or update listings, manage availability, handle bookings, and respond to user feedback, ensuring accurate, real-time information for users and reliable service delivery.

By addressing the limitations of existing travel systems and incorporating essential nonfunctional and functional requirements such as usability, authentication, data integrity, performance, availability, and security, Travel Pro ensures a reliable and efficient operation that benefits both platform administrators and travelers.

In essence, the Travel Pro project stands as a testament to the integration of modern technology with user-focused design principles, redefining digital travel management, enhancing convenience, and fostering greater user satisfaction and platform trust.

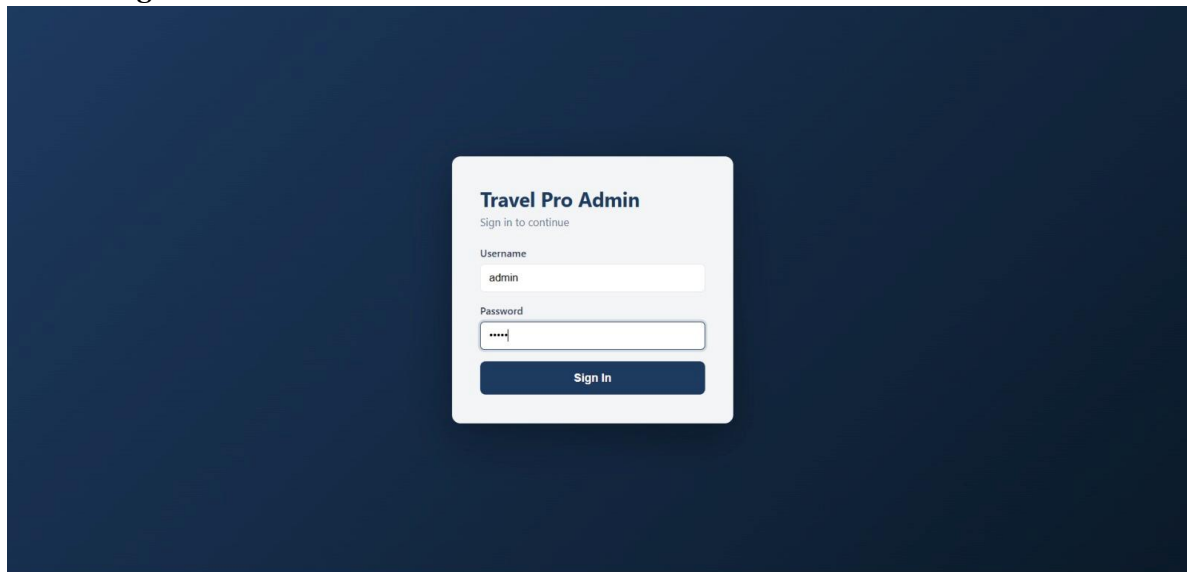
APPENDIX

GANTT CHART



SCREENSHOTS

Admin Login



The screenshot shows the 'Travel Pro Admin' login interface. It features a dark blue background with a central white login card. The card has the title 'Travel Pro Admin' and a subtitle 'Sign in to continue'. Below this, there are input fields for 'Username' (containing 'admin') and 'Password' (masked with dots). A 'Sign In' button is positioned at the bottom of the card.

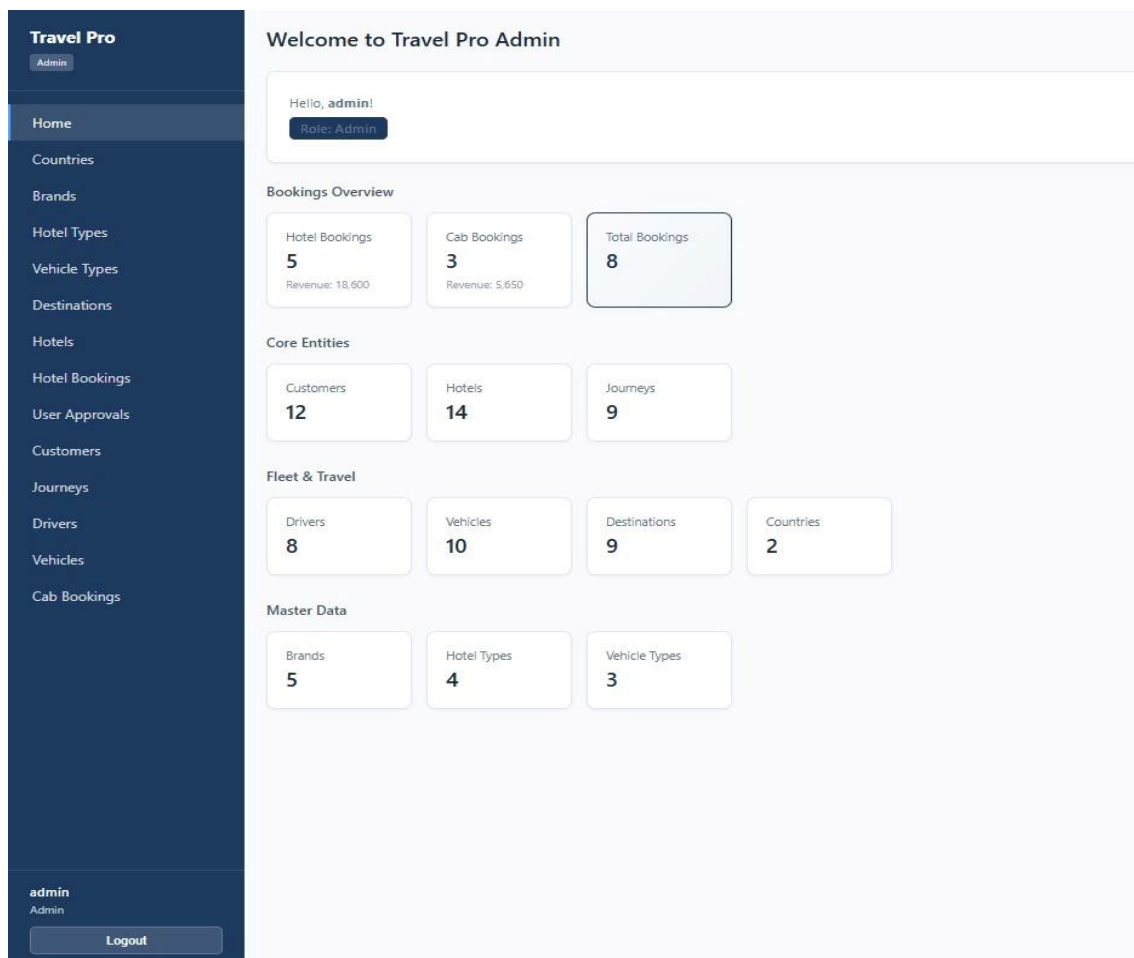
Travel Pro Admin
Sign in to continue

Username
admin

Password

Sign In

Admin Dashboard



The screenshot displays the 'Travel Pro Admin' dashboard. On the left is a dark blue sidebar with the 'Travel Pro' logo and 'Admin' role. The sidebar menu includes 'Home', 'Countries', 'Brands', 'Hotel Types', 'Vehicle Types', 'Destinations', 'Hotels', 'Hotel Bookings', 'User Approvals', 'Customers', 'Journeys', 'Drivers', 'Vehicles', and 'Cab Bookings'. At the bottom of the sidebar, the user 'admin' is logged in, with a 'Logout' button.

The main content area is titled 'Welcome to Travel Pro Admin' and includes a greeting 'Hello, admin!' and a 'Role: Admin' badge. Below this, the dashboard is organized into several sections:

- Bookings Overview:** Three cards showing 'Hotel Bookings' (5, Revenue: 18,600), 'Cab Bookings' (3, Revenue: 5,650), and 'Total Bookings' (8).
- Core Entities:** Three cards showing 'Customers' (12), 'Hotels' (14), and 'Journeys' (9).
- Fleet & Travel:** Four cards showing 'Drivers' (8), 'Vehicles' (10), 'Destinations' (9), and 'Countries' (2).
- Master Data:** Three cards showing 'Brands' (5), 'Hotel Types' (4), and 'Vehicle Types' (3).

Hotel Types

Travel Pro

Admin

Home

Countries

Brands

Hotel Types

Vehicle Types

Destinations

Hotels

Hotel Bookings

User Approvals

Customers

Journeys

Drivers

Vehicles

Cab Bookings

Admin

Admin

Logout

Hotel Types

Add Hotel Type

Filter

Filter by name...

#	Name	Actions
1	Resort	Edit Delete
2	Five Star	Edit Delete
3	Three Star	Edit Delete
4	2 Star	Edit Delete

Vehicle Types

Travel Pro

Admin

Home

Countries

Brands

Hotel Types

Vehicle Types

Destinations

Hotels

Hotel Bookings

User Approvals

Customers

Journeys

Drivers

Vehicles

Cab Bookings

Admin

Admin

Logout

Vehicle Types

Add Vehicle Type

Filter

Filter by name...

#	Name	Actions
1	4 Seater	Edit Delete
2	7 seater	Edit Delete
3	2 seater	Edit Delete

Destinations

Travel Pro

Admin

Home

Countries

Brands

Hotel Types

Vehicle Types

Destinations

Hotels

Hotel Bookings

User Approvals

Customers

Journeys

Drivers

Vehicles

Cab Bookings

admin

Admin

Login

Add Destination

Search

Country

Name, description, country.

All

#	Photo	Name	Country	Actions	
1		Munnar	India	Edit	Delete
2		Trivandrum	India	Edit	Delete
3		Vagamon	India	Edit	Delete
4		Thekkala	India	Edit	Delete
5		Punalur	India	Edit	Delete
6		Gavi	India	Edit	Delete
7		Kuttanad	India	Edit	Delete
8		Fort Kochi	India	Edit	Delete
9		Wayanad	India	Edit	Delete

User pages

User login page

Travel Plus

[Home](#)
[Travels](#)
[Destinations](#)
[Hotels](#)
[Cabs](#)
[About](#)
[Contact](#)
[Login](#)

Sign In

Enter your credentials to access your account

Username / Email

kailas123@gmail.com

Password

☒ Remember me
 [Forgot Password?](#)

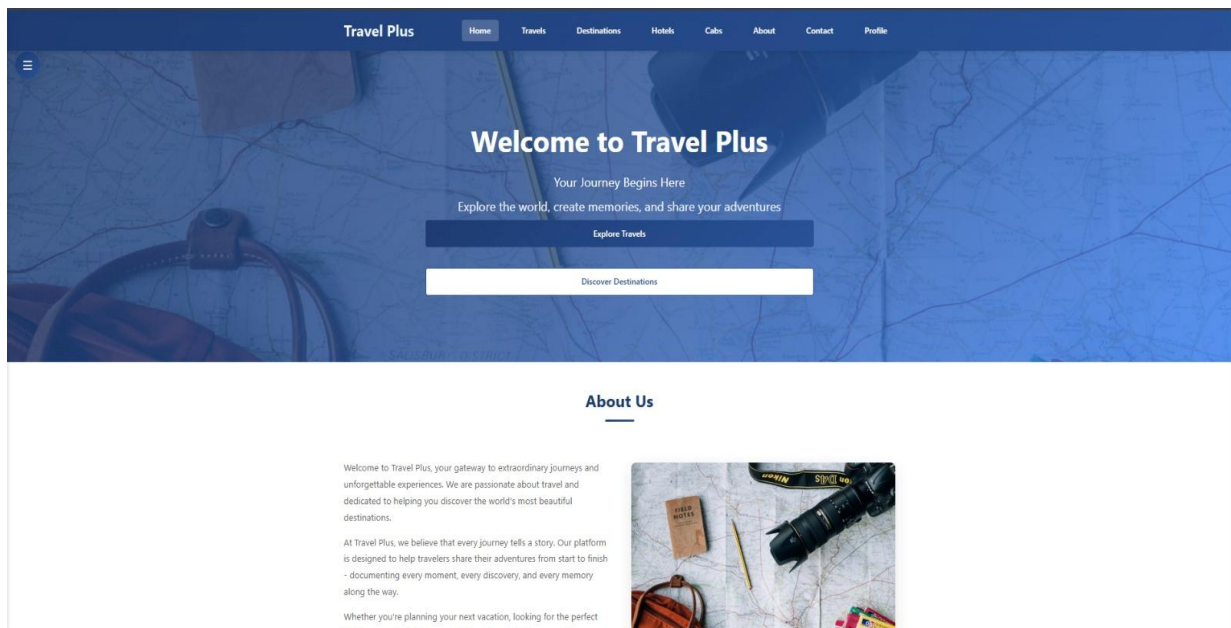
Sign In

Don't have an account? [Sign Up](#)

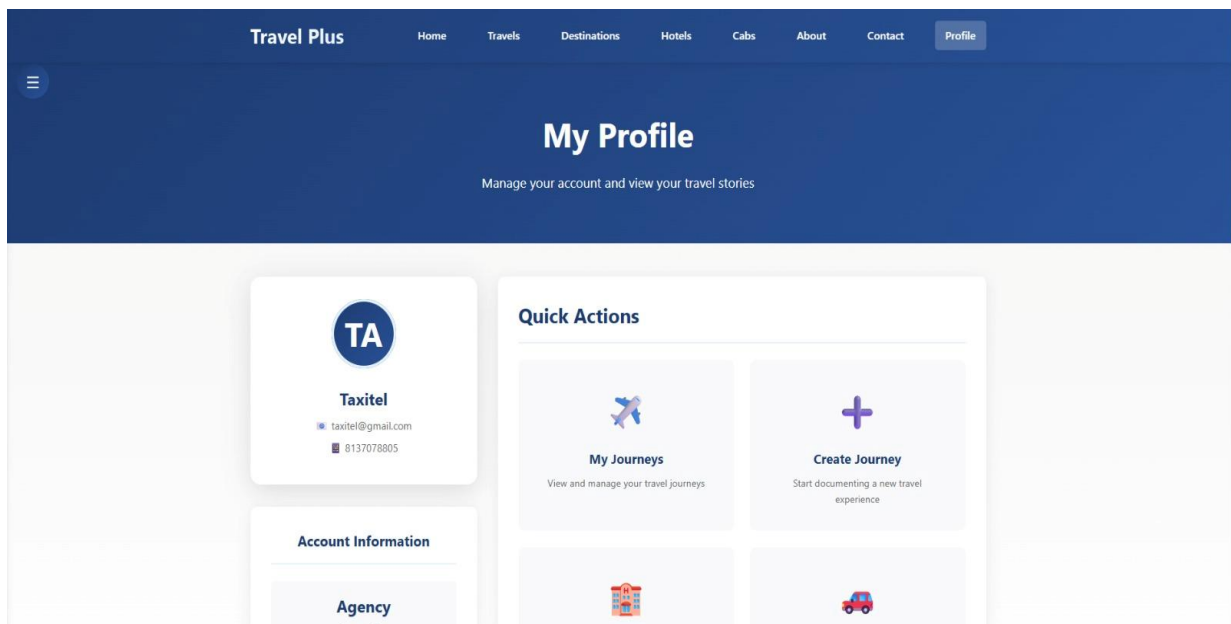
Start Your Journey

Explore amazing destinations and create unforgettable memories

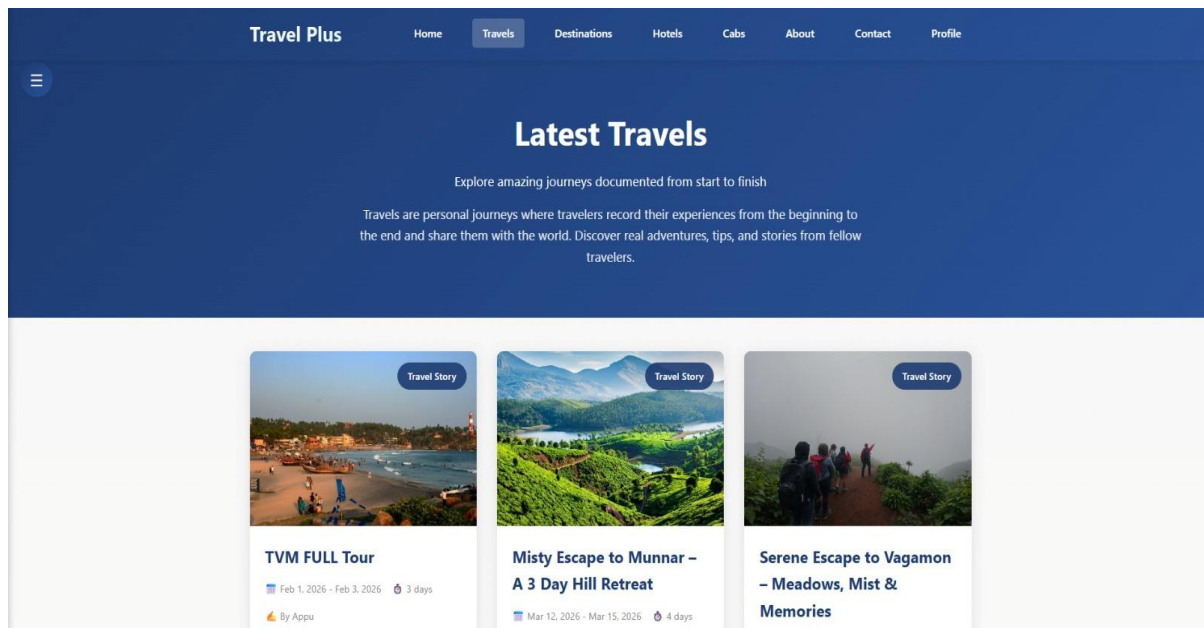
Homepage



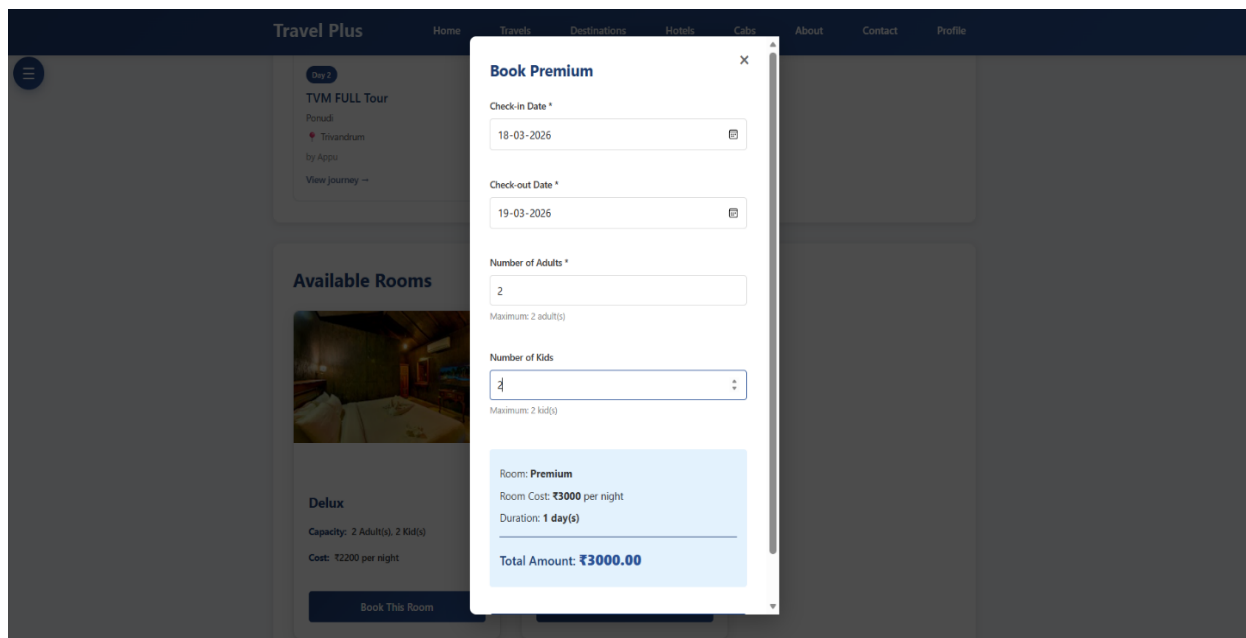
User profile page



Travel Page



Hotel Booking Page



Cab Booking Page

Travel Plus

HomeTravelsDestinationsHotelsCabsAboutContactProfile

☰

Toyota vellfire

Year: 2024

Driver: Ravi (889878874)

ACGPSBluetoothWi-Fi

Music SystemCameraSensors

LCDSafety Features

7 SeatsPetrolAuto

Black

₹50 per ride

Book Now

Book Toyota vellfire

Pickup Location

kutichal

Drop Location

kattakada

Destination

Trivandrum

Distance (km)

12

Pickup Date

19-03-2026

Pickup Time

05:42

Number of Passengers

1

Travel Plus

Your journey begins here. Explore the world with us and create unforgettable memories.

Qu

Ho

Tr

De

Ho

Ca

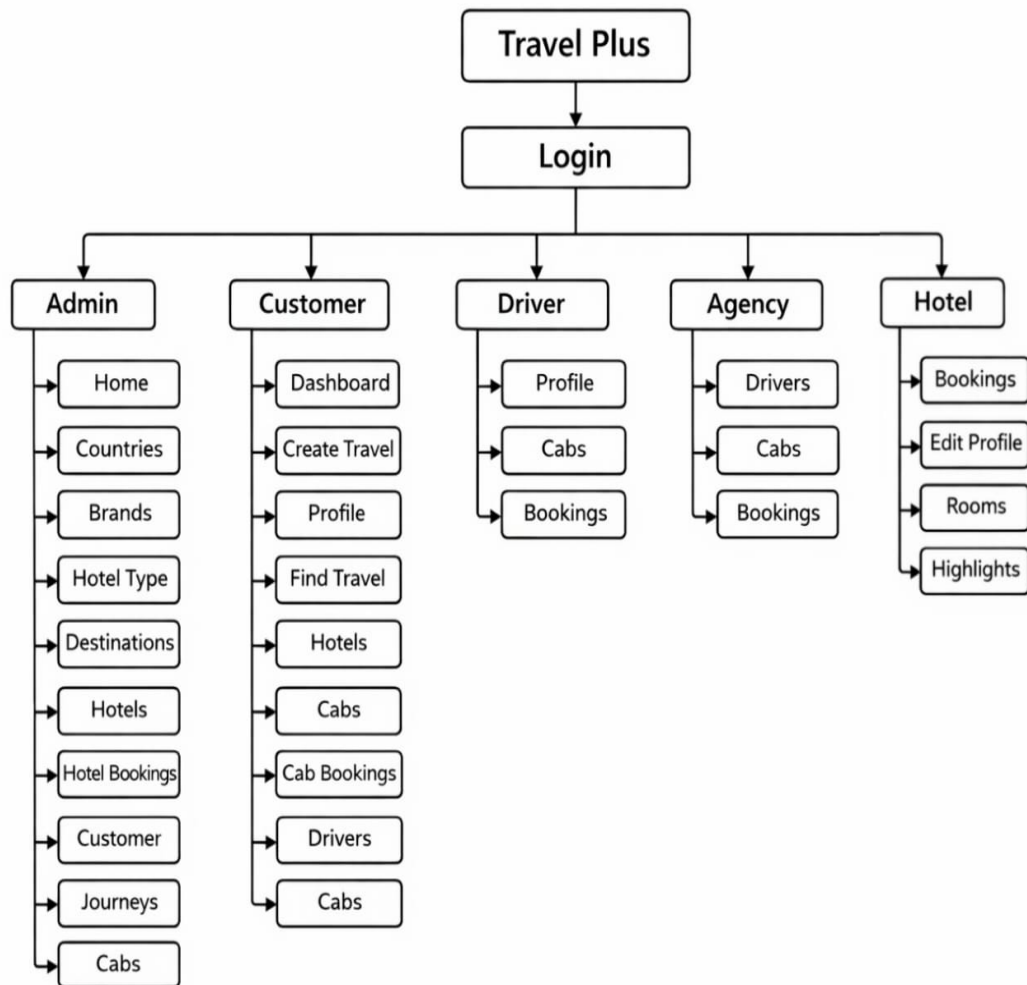
Contact Info

Email: info@travelplus.com

Phone: +1 234 567 8900

Address: 123 Travel Street, Tourism City

MENU TREE



Meeting Minutes

Date: 16-07-2025

Place: Christ Nagar College, Maranalloor

Members: Abhinav Rajeev A, Ashiq S, Goutham Hareesh, Kailas Jith V A, Sudeep S

Discussion: Discussed about the topic.

Date: 18-07-2025

Place: Christ Nagar College, Maranalloor

Members: Abhinav Rajeev A, Ashiq S, Goutham Hareesh, Kailas Jith V A, Sudeep S

Discussion: Travel Management System finalized.

Date: 21-07-2025

Place: Christ Nagar College, Maranalloor

Members: Abhinav Rajeev A, Ashiq S, Goutham Hareesh, Kailas Jith V A, Sudeep S

Discussion: Conducted research on existing system and its drawbacks.

Date: 23-07-2025

Place: Christ Nagar College, Maranalloor

Members: Abhinav Rajeev A, Ashiq S, Goutham Hareesh, Kailas Jith V A, Sudeep S

Discussion: Discussed about the various modules and functionalities to be included in the project.

Date: 25-07-2025

Place: Christ Nagar College, Maranalloor

Members: Abhinav Rajeev A, Ashiq S, Goutham Hareesh, Kailas Jith V A, Sudeep S

Discussion: Categorized the project into four modules.

Date: 04-08-2025

Place: Christ Nagar College, Maranalloor

Members: Abhinav Rajeev A, Ashiq S, Goutham Hareesh, Kailas Jith V A, Sudeep S S

Discussion: Documentation started.

Date: 07-08-2025

Place: Christ Nagar College, Maranalloor

Members: Abhinav Rajeev A, Ashiq S, Goutham Hareesh, Kailas Jith V A, Sudeep S S

Discussion: Started design of webpages.

Date: 09-08-2025

Place: Christ Nagar College, Maranalloor

Members: Abhinav Rajeev A, Ashiq S, Goutham Hareesh, Kailas Jith V A, Sudeep S S

Discussion: Design of project was completed.

Date: 10-08-2025

Place: Christ Nagar College, Maranalloor

Members: Abhinav Rajeev A, Ashiq S, Goutham Hareesh, Kailas Jith V A, Sudeep S S

Discussion: Coding for Admin module started.

Date: 11-08-2025

Place: Christ Nagar College, Maranalloor

Members: Abhinav Rajeev A, Ashiq S, Goutham Hareesh, Kailas Jith V A, Sudeep S S

Discussion: Coding for user module started.

Date: 08-09-2025

Place: Christ Nagar College, Maranalloor

Members: Abhinav Rajeev A, Ashiq S, Goutham Hareesh, Kailas Jith V A, Sudeep S S

Discussion: Testing started for completed modules.

Date: 10-09-2025

Place: Christ Nagar College, Maranalloor

Members: Abhinav Rajeev A, Ashiq S, Goutham Hareesh, Kailas Jith V A, Sudeep S S

Discussion: Module-wise functionalities completed.

Date: 11-10-2025

Place: Christ Nagar College, Maranalloor

Members: Abhinav Rajeev A, Ashiq S, Goutham Hareesh, Kailas Jith V A, Sudeep S S

Discussion: Database created.

Date: 16-11-2025

Place: Christ Nagar College, Maranalloor

Members: Abhinav Rajeev A, Ashiq S, Goutham Hareesh, Kailas Jith V A, Sudeep S S

Discussion: Backend started.

Date: 28-01-2026

Place: Christ Nagar College, Maranalloor

Members: Abhinav Rajeev A, Ashiq S, Goutham Hareesh, Kailas Jith V A, Sudeep S S

Discussion: Documentation discussion.

Date: 02-02-2026

Place: Christ Nagar College, Maranalloor

Members: Abhinav Rajeev A, Ashiq S, Goutham Hareesh, Kailas Jith V A, Sudeep S S

Discussion: Front End coding started.

Date: 09-02-2026

Place: Christ Nagar College, Maranalloor

Members: Abhinav Rajeev A, Ashiq S, Goutham Hareesh, Kailas Jith V A, Sudeep S S

Discussion: Backend coding completed.

Date: 11-02-2026

Place: Christ Nagar College, Maranalloor

Members: Abhinav Rajeev A, Ashiq S, Goutham Hareesh, Kailas Jith V A, Sudeep S S

Discussion: Coding completed.

Date: 19-02-2026

Place: Christ Nagar College, Maranalloor

Members: Abhinav Rajeev A, Ashiq S, Goutham Hareesh, Kailas Jith V A, Sudeep S S

Discussion: Backend and database linking completed.

Date: 11-03-2026

Place: Christ Nagar College, Maranalloor

Members: Abhinav Rajeev A, Ashiq S, Goutham Hareesh, Kailas Jith V A, Sudeep S S

Discussion: Testing completed.

Date: 13-03-2026

Place: Christ Nagar College, Maranalloor

Members: Abhinav Rajeev A, Ashiq S, Goutham Hareesh, Kailas Jith V A, Sudeep S S

Discussion: Verification and validation process done and completed project.

Date: 16-03-2026

Place: Christ Nagar College, Maranalloor

Members: Abhinav Rajeev A, Ashiq S, Goutham Hareesh, Kailas Jith V A, Sudeep S S

Discussion: Documentation completed.

BIBLIOGRAPHY

BOOK REFERENCES

1. *REACT: The Road To Learn React*; By Robin Wieruch
2. *Node.js: Mastering Node.js*; By Sandro Pasquali
3. *MySQL: Learning MySQL :Get Handle Your Data*; By Vinius

WEBSITE REFERENCE

1. <https://www.wikipedia.com/software/carrentalsystem>
2. <https://github.com/topics/cab>
3. <https://www.royalbrothers.com>
4. <https://www.rapido.rentals>